

B.A.Sc. Thesis



Division of Engineering Science
UNIVERSITY OF TORONTO

Developing Explainable AI models for DDoS Network Intrusion Detection Systems

by
Ryan Chen

Submitted to the Division of Engineering Science
in partial fulfillment of the requirements for the degree of
BACHELOR OF APPLIED SCIENCE IN ENGINEERING SCIENCE
at the
UNIVERSITY OF TORONTO
May 2024

Authored by: Ryan Chen
Division of Engineering Science
April 12, 2024

Certified by: Kuei J. Sun
Professor of Computer Science, Thesis Supervisor

Developing Explainable AI models for DDoS Network Intrusion Detection Systems

by

Ryan Chen

Submitted to the Division of Engineering Science
on April 12, 2024 in partial fulfillment of the requirements for the degree of

BACHELOR OF APPLIED SCIENCE IN ENGINEERING SCIENCE

ABSTRACT

In the domain of cybersecurity, trust and transparency are foundational elements crucial for ensuring system integrity and thwarting malicious actors. However, the rapid advancement of AI, particularly in the form of black box systems like deep neural networks (DNNs), has introduced challenges to maintaining this trust. Current methods for explaining black box AI models often require substantial resources and are applied post hoc, resulting in static explanations that may not reflect real-world conditions. This research focuses on the development and application of explainable artificial intelligence (xAI) methods during model training to produce inherently transparent models. Specifically, the study aims to provide a clear rationale for predictions generated by an Intrusion Detection System (IDS) trained to detect Distributed Denial of Service (DDoS) attacks. By integrating explanation methods such as Shapley values into the training process, the research aims to generate knowledge graph interpretations of the model which can be compared to a researcher constructed knowledge graph to provide ad hoc explanations that tune the model's explainability according to the researcher's understanding of feature interactions.

Thesis supervisor: Kuei J. Sun

Title: Professor of Computer Science

Acknowledgments

A word of thanks to my supervisor, Professor Kuei (Jack) Sun, for his guidance on computer networking and communications. This thesis would not have been possible without the insights and background knowledge gained from our valuable discussions and 9pm meetings.

Special thanks to Professor Michael Guerzhoy for aiding in the initial establishment of research direction and project feasibility with respect to the field of machine learning, and Courtney Gibson for his consulting on matters regarding network security and DDoS attacks.

Contents

Title page	2
Abstract	3
Acknowledgments	4
List of Figures	7
List of Tables	8
1 Introduction	9
1.1 AI is a Black Box	9
1.2 A Light in the Box	9
1.3 Explainability Ad Hoc	10
2 Background	11
2.1 Distributed Denial of Service	11
2.2 Intrusion Detection Systems	12
2.3 Popular Explanation Tools	14
3 Designing an Innately Transparent IDS	15
3.1 Experimental Objectives	15
3.2 Experimental Constraints	16
3.3 The CIC-DDoS2019 Dataset	17
3.4 Data Processing	18
3.5 IDS Design	20
3.5.1 Anomaly Detection IDS	20
3.5.2 Signature Detection IDS	21
3.6 Developing Explanations Innately	22
3.6.1 Knowledge Graphs	22
3.6.2 SHAP Loss	28
3.6.3 SHAP Graph Edit Distance	29
4 Experiment Takeaways	31
4.1 Model Performance	31
4.1.1 Anomaly Detection Performance	31

4.1.2	Signature Detection Performance	32
4.2	Model Explanations	34
4.2.1	Anomaly Detection Explanation	34
4.2.2	Signature Detection Explanation	36
5	Closing Remarks	40
5.1	Final Notes	40
	References	41
A	Code listing	43
A.1	Training Code	43
A.2	Dataset Code	50
A.3	Anomaly Detection Model Code	53
A.4	Signature Detection Model Code	54
B	Additional Machine Learning Content	56
B.1	Convolutional Neural Networks	56
B.2	CICDDoS2019 Features	56
C	Relevant Explainable Machine Learning Material	60
C.1	Local Interpretable Model Agnostic Explanations	60
C.2	Shapley Values	61
C.3	SHapley Additive exPlanations	62
C.4	eXplainable Neural-Symbolic Learning	63

List of Figures

2.1	DDoS Taxonomy [2]	12
2.2	Example 1D CNN architecture for IDS employed by Meliboyev et al. in <i>1D CNN based network intrusion detection with normalization on imbalanced data</i> [5]	13
3.1	Definition of Accuracy	15
3.2	CIC-DDoS2019 Testbed architecture for data analysis and generation [2]	17
3.3	Anomaly Detection Model Architecture	21
3.4	Signature Detection Model Architecture	21
3.5	Binary Classification Knowledge Graph	23
3.6	Topk Signature Detection Knowledge Graph, $k = 5$	25
3.7	Pruned Signature Detection Knowledge Graph	26
4.1	Anomaly Detection Model Accuracies	32
4.2	Anomaly Detection Model Confusion Matrices @ 1050 Iterations	32
4.3	Signature Detection Model Accuracies	33
4.4	Signature Detection Model Confusion Matrices @ 500 Epochs and 100 Epochs	33
4.5	Signature Detection Model Benign Class Accuracies	34
4.6	Anomaly Detection Model SHAP Graph Edit Distances in Training	35
4.7	Anomaly Detection Model Shapley Values for Benign Predictions	36
4.8	Signature Detection Model SHAP Graph Edit Distances in Training	37
4.9	Signature Detection Model Shapley Values for Benign Predictions	38

List of Tables

3.1	Processed Attack Frequencies	19
3.2	Attack Frequencies after Reservoir Sampling from Processed Frequencies . .	20
3.3	Anomaly Detection Knowledge Graph Benign Features	23
3.4	Anomaly Detection Knowledge Graph Malicious Features	24
3.5	Signature Detection Knowledge Graph Features	26
B.1	Raw Dataset Features and Descriptions	57

Chapter 1

Introduction

1.1 AI is a Black Box

Trust is one of the most important foundational elements in the realm of cybersecurity, it stands as a core piece of integrity, a property of all secure systems. Integrity in this context demands a level of transparency at all layers of a system, such that no malicious actor can perpetrate their activities undetected and unrestricted. Detection and prevention are the two primary fields of cybersecurity, with the former being a classification task that has seen its fair share of successful automation attempts. Rapid developments in AI and the prominent rise of complex black box systems like deep neural networks (DNN) have introduced a gap in trust and transparency, as they have struggled to keep up with the rapid improvement and spread of the technology. Instead, explainability for black box systems has taken a backseat in favour of researching and developing more complex and resource intensive models to increase raw performance metrics.[1]

1.2 A Light in the Box

Current state of the art methods to explain black box AI models require heavy allocation of compute and human resources to provide full coverage of a system's operation. These methods are also performed post hoc to the model's initial training, which may delay the development process. This post hoc explanation converts the black box model into a white box model; explainable but static. The current explanation techniques are only relevant to the model and its parameters at a given point in time when the explanation technique is applied. Security researchers have no way of assessing a model's explainability during the training process and cannot influence the model in any way during training to become more explainable. This gap is further accentuated by the field's researchers reliance on outdated datasets with unbalanced class labels to create trivial model explanations that no longer reflect real world network conditions.[2]

1.3 Explainability Ad Hoc

The integration of a black box AI into a system poses new threats, as vulnerabilities are created when false assumptions are made by security professionals about a system's operations. This thesis centers on the exploration and application of explainable artificial intelligence (xAI) methods in model training to produce inherently transparent models which researchers can control the degree of explainability ad hoc. The primary objective is to outline a clear rationale for the predictions generated by the Intrusion Detection System (IDS) model trained to detect Distributed Denial of Service (DDoS) attacks in response to specific inputs during training. By dissecting the intricacies of the model's classification process, the thesis will aim to create an acceptable level of transparency by unveiling the reasoning behind predictions, and discovering potential exploitable weaknesses in the reasoning of black box models. This will pave the way for a comprehensive examination of new potential threats and attack vectors. The ultimate goal is to propose corrective measures and enhancements that bolster the model's overall security and utility in the hands of security professionals.

Chapter 2

Background

2.1 Distributed Denial of Service

DDoS attacks pose a significant threat in the cybersecurity landscape, targeting the accessibility of services between hosts and their clients. These attacks operate by overwhelming the host network with an unprecedented volume of requests, surpassing the target network's bandwidth capacity. [3] A key tactic employed by attackers involves the creation of a botnet—a network of compromised machines acting as zombie devices. The utilization of multiple machines enhances the scale and effectiveness of the attack while making detection more challenging. The incorporation of legitimate third-party components in the transmission of packets from zombie devices to reflector servers adds a layer of complexity to the detection process. [3]

DDoS attacks manifest in two primary forms: reflection attacks and exploitation attacks. Reflection attacks exploit handshake protocols such as Transmission Control Protocol (TCP) or User Datagram Protocol (UDP), where the response size dwarfs the initial request. In this scenario, botnet devices spoof the IP address of the target network, sending requests to authentic reflector servers. [3] This prompts the reflector servers to inundate the target network with responses, aggravating the impact of the attack. On the other hand, exploitation attacks manipulate network services by inducing unexpected behaviors or disrupting network handshake protocols like TCP and UDP. This leads to a buildup of requests, causing the targeted system's service to degrade to the point of inactivity, bypassing the need for an overwhelming volume of requests beyond the system's bandwidth capacity. [3]

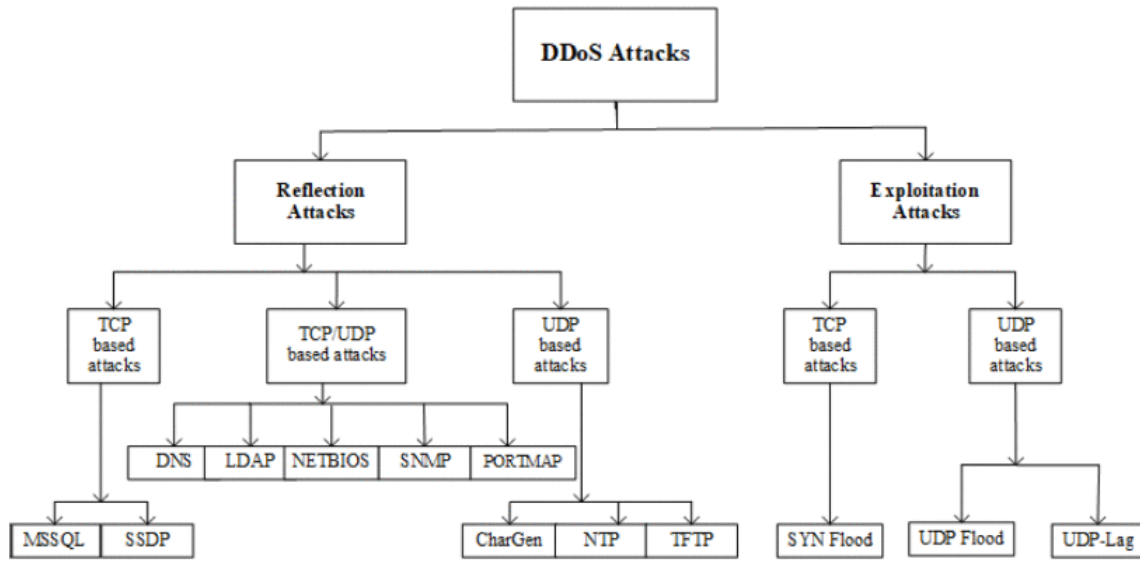


Figure 2.1: DDoS Taxonomy [2]

The consequences of successful DDoS attacks are severe, resulting in both immediate and prolonged service interruptions, coupled with the creation of large data backlogs. The intricacies of these attack vectors underscore the critical need for advanced cybersecurity measures, including explainable AI, to understand, detect, and mitigate the evolving nature of DDoS threats.

2.2 Intrusion Detection Systems

Intrusion Detection Systems (IDS) play a pivotal role in cybersecurity, functioning as either software or hardware solutions designed to monitor, analyze, and identify anomalies, suspicious, or malicious behaviors within device communications. This proactive monitoring enables system administrators to plan and take appropriate actions in response to potential security threats.

Two fundamental implementations of IDS exist: the Network IDS (NIDS) and Host-based IDS (HIDS). NIDS focuses on monitoring network traffic by analyzing information transmitted into a local network from external sources through data packets. On the other hand, HIDS operates on a local machine, scrutinizing file system activities for any signs of suspicious behavior. The thesis will use IDS to strictly refer to network IDS, regarding the subject matter.

IDS can be further classified based on their detection methodologies, with two prominent categories being signature-based and anomaly-based detection. Signature-based detection involves monitoring network patterns associated with known malicious activities, providing a reactive approach to security. In contrast, anomaly-based detection functions inversely by flagging activities that deviate from established patterns of regular and intended network usage, allowing for a more proactive threat identification.

However, signature-based detection systems have limitations, being less robust as they can only respond to known threats. Continuous updates and frequent collection of new data are imperative for these systems to remain effective and secure. On the other hand, anomaly-based detection systems exhibit greater adaptability, capable of handling unknown threats, making them particularly adept at addressing sophisticated attacks such as DDoS. This distinction positions anomaly-based detection as a more resilient and versatile approach in the realm of intrusion detection, especially in the context of evolving cybersecurity challenges.

Several prior works [1], [4], [5] have found a considerable degree of success using convolutional neural networks (CNN) for the purposes of intrusion detection with tabular data. Their architecture consists of a series of convolutional filters which extract sets of features from their input and a series of fully connected neuron layers which predict the class to which the extracted features most likely belong.[6] CNNs excel at feature extraction with a sense of feature locality, and are often applied to image processing and computer vision tasks using two-dimensional convolutions. This locality based feature extraction can also be leveraged for formatted, ordered tabular data in the one-dimensional case, allowing for improved detection by subsequent fully connected dense layers. Given proof of concept, relevance, and results from prior works, the CNN black box is an excellent candidate on which to attempt innate explanation methods.

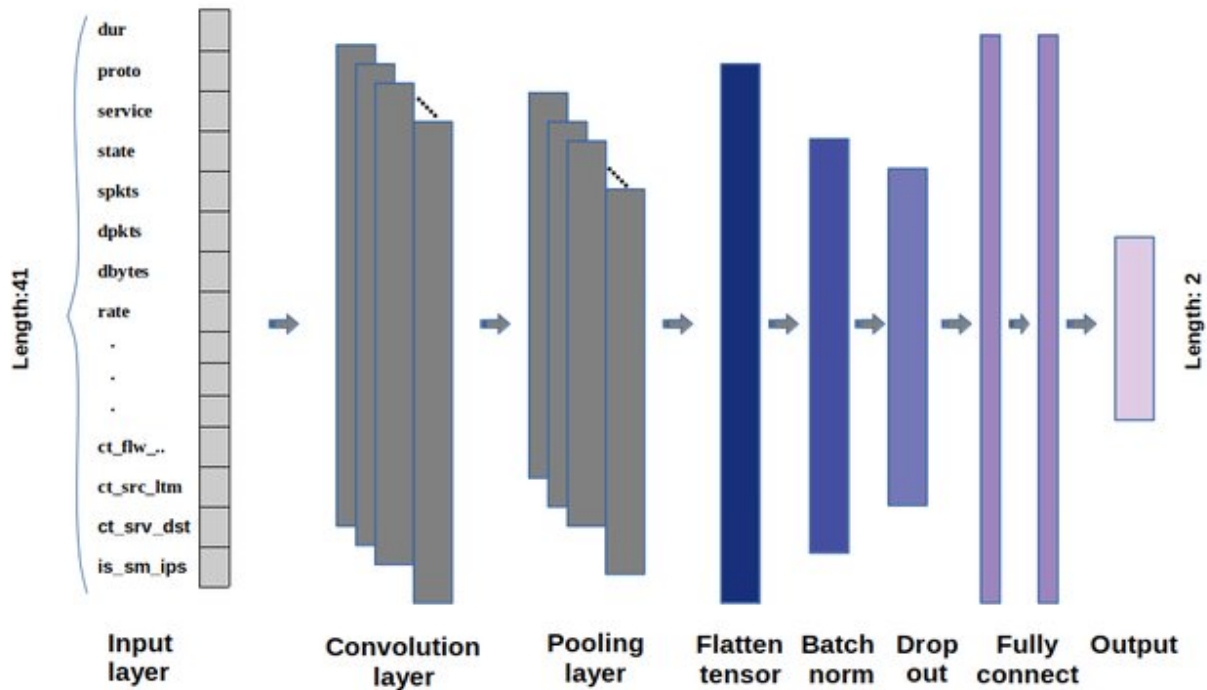


Figure 2.2: Example 1D CNN architecture for IDS employed by Meliboyev et al. in *1D CNN based network intrusion detection with normalization on imbalanced data* [5]

2.3 Explanation Tools

The field of xAI is shifting towards model agnostic methods with powerful local explanations. Implying a need for methods which are not uniquely dependent on model architecture to explain predictions, and the ability to explain singular predictions. The ability to explain singular predictions is powerful, as it allows researchers to investigate unexpected behaviours with their models with a higher degree of fidelity compared to investigating the entirety of the model’s prediction space. Additionally local explainability can be aggregated and averaged over a large sample of predictions to reach a global explanation from the ground up, with some additional processing.

One such powerful method is borrowed from game theory, Shapley values[7]. Traditionally, Shapley values measure the contribution of a single player towards a goal in a cooperative game given they are part of a coalition of other players. Repurposed for machine learning, it determines the contribution of a single feature (player) towards a predicted class label (goal). This provides a solid foundation for decrypting black box models into white box models. Shapley values are used as a post hoc evaluation tool given the exponential computational complexity when calculating Shapley values over a large set of predictions to approximate a global explanation. By reducing the sample size of Shapley Value calculations, we can trade the accuracy of the contribution measurement for some computational efficiency in training for noisy ad hoc explanations.

Another method of approximating Shapley values is with the use of the Deep Learning Important FeaTures algorithm (DEEPLIFT).[8] DEEPLIFT approximates Shapley values by measuring the differences of activation functions during back propagation to trace back contributions from each feature at every layer of a DNN. By combining the use of this algorithm and a reasonably safe reduction in sample size for calculating Shapley values, it may be feasible to implement a Shapley Value-based explanation tool in the training process of an machine learning based IDS model.

Chapter 3

Designing an Innately Transparent IDS

3.1 Experiment Objectives

The primary objective of this study is to implement a method capable of providing ad hoc explanations during the development and training process of machine learning based IDS. This should elevate the model beyond a white box to an interactive model which researchers can tune the explainability of with respect to their own understanding of how features interact with one another and contribute to a prediction label. This can be accomplished via a modified metric used by N. Díaz Rodríguez et al. in a neuro-symbolic learning paper, a graph edit distance between two knowledge graphs.[9] One knowledge graph would be proposed by the researcher seeking to understand the model, and this would be compared to a knowledge graph generated by the IDS model itself based on machine learning explainability methods. This allows for a direct comparison between how the model interprets the data and how the researcher interprets the data, with criteria stating that a lower graph edit distance being better, meaning the model’s reasoning of the data approaches that of the researcher’s.

$$Accuracy = \frac{TruePositive + TrueNegative}{(TruePositive + FalsePositive + TrueNegative + FalseNegative)} \quad (3.1)$$

Figure 3.1: Definition of Accuracy

The secondary objective of the experiment is to attain a 99% accuracy rate in anomaly detection for models trained using the new explanation method, aiming to align with the model performance of previous works in the field.[1], [4], [5] This target serves as a critical benchmark, indicating that explainability methods do not greatly impede model performance and generalizability in most use cases. The significance of this benchmark with respect to the primary objective stems from the fact that the model must make correct predictions to possess a human-interpretable explanation regarding the subject matter. This benchmark will be ignored for signature detection models, as the task is substantially more difficult with multiple classes. In addition to the more difficult nature of the task, a general disagreement between prior work exists for what an is obtainable model prediction accuracy rate. Instead,

it is important look at the accuracy for a single class of benign packets, effectively regressing the signature detection benchmark to that of an anomaly detection benchmark, since differentiation between benign and malicious packet flows was still the most important task. The metric tolerates misclassifications between the signatures of malicious packet flows.

3.2 Experiment Constraints

Regarding constraints for machine learning tasks, the primary roadblock is usually the available compute power for training and deployment of the model. Navigating experiment constraints related to computing resources is a pivotal aspect of ensuring the execution of the experiment and usability of the work. The experimental process will leverage on hand compute resources available, including hardware such as a single AMD Ryzen 7 3800XT 8-Core CPU and NVIDIA GeForce RTX 3070 8GB VRAM GPU. While these components offer substantial computational power and should be valid for this use case, they are by no means capable of training state-of-the-art (SotA) competitive models which may see greater raw performance across the board. Experiment execution must balance the workload effectively between CPU and GPU resources, in order to optimize performance while mitigating bottlenecks which may delay research progression. Considerations such as memory allocation and parallel processing capabilities play a critical role in maximizing efficiency, as a GPU with 8GB of VRAM may not be sufficient in holding enough data in memory to fully train the model. By strategically managing computing resources in combination with the large quantities of data, experiments can be executed efficiently, facilitating rigorous exploration and evaluation of ad hoc explainability methods in machine learning models within the confines of available hardware.

With the computational constraints in place, experiment constraints concerning machine learning model complexity follows suit. This constraint serves to balance the intricacy of model architecture with computational feasibility. While more complex models may potentially capture finer patterns in the data, they often come with increased computational demands and the risk of overfitting. Conversely, simpler models offer ease of application for interpretability methods and computational efficiency but might struggle to capture the nuances present in the data. It is paramount to carefully assess the trade-offs between model complexity and performance, considering factors such as dataset size, computational resources, and the interpretability requirements of the problem at hand. Given the nature of this thesis and the available compute power, a simpler model should suffice in fulfilling the general purposes of modifying and testing interpretability methods ad hoc in the training process. Striking the appropriate balance for the given task is crucial for ensuring the model's effectiveness while avoiding unnecessary complexity that could impede its practical applicable utility, and generalizability. As mentioned, the criteria for this constraint will prioritize minimizing model footprint and creating compact models which ease interpretability tasks. This can be measured in the number of layers for the neural net architecture, and the number of trainable parameters possessed by the model.

3.3 The CIC-DDoS2019 Dataset

The CIC-DDoS2019 dataset, curated by the Canadian Institute for Cybersecurity (CIC), stands as the latest comprehensive resource for the study of DDoS cyber attacks. Unique to this dataset is the implementation of a B-Profile system to generate data reflective of real human net traffic. B-Profiling is designed to characterize the abstract behavior of human interactions and generate naturalistic benign background traffic within a controlled testbed.[2], [10] The B-Profiling mechanism involves collecting client traffic, constructing statistical profiles based on individual behaviors, and clustering user profiles through the KMeans algorithm, considering both user behavior and placement in network usage distributions. The clusters provide a set of generic human net behaviour for training purposes.[2]

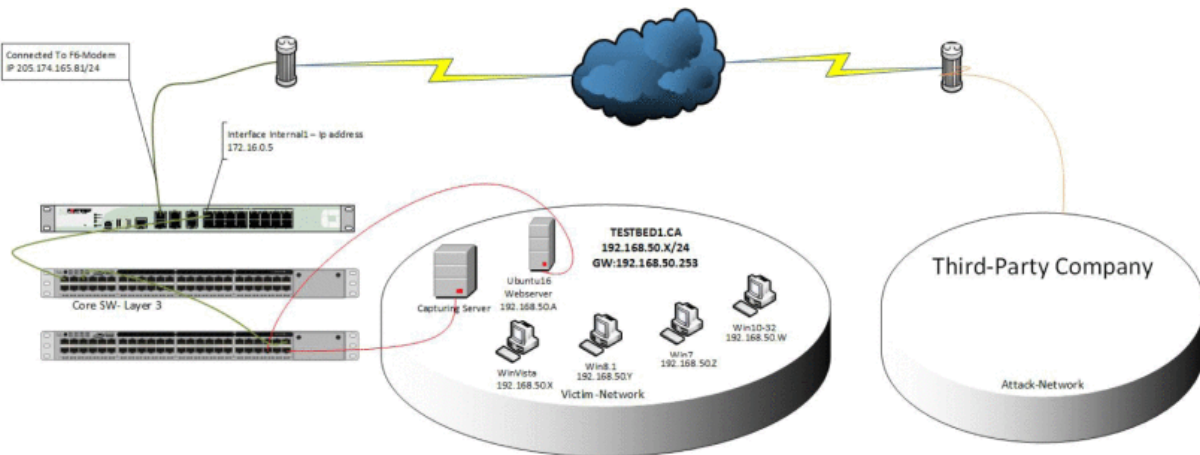


Figure 3.2: CIC-DDoS2019 Testbed architecture for data analysis and generation [2]

The dataset is derived from controlled attacks executed on an Ubuntu 16.04 Web Server, fortified by a Fortinet Firewall, and integrated with benign Windows Vista, 7, 8.1, and 10 machines as integral components of the network architecture. The analysis of network traffic was conducted using CICFlowMeter-V3, which also facilitated the labeling of the data. Various DDoS attacks were intentionally induced, including PortMap, NetBIOS, LDAP, MSSQL, UDP, UDP-Lag, SYN, NTP, DNS, SNMP, and SSDP.[2] Although WebDDoS was part of the experimental setup, insufficient data points were collected for it to be considered in the analysis.[4]

The dataset provides a rich pool of information, encompassing 88 distinct traffic features recorded using CICFlowMeter-V3, derived from both packet data (Pcaps) and device (Windows and Ubuntu) event logs. This extensive and carefully collected dataset serves as a valuable resource for training and evaluating IDS, and particularly helpful in developing explainable AI models for heightened comprehension and mitigation of DDoS threats.[2]

3.4 Data Processing

Data processing for the CICDDoS2019 dataset consisted of several steps including the following:

- Removal of features with only 0 or non-numeric data values
- Removal of data entries with non-numeric, infinite, and negatively infinite values
- Removal of data entries belonging to the WebDDoS attack class
- Removal of duplicate data entries
- Scalar normalization on each feature
- Balance the ratio of ground truth labels by reservoir sampling the size of the smallest class from each label class

The steps for data processing closely follow that of Akgun et al.[4] The first step was to remove the features which contribute nothing to the output as a result of an entirely uniform collection of 0 values, or incomputable values such as text strings or null/missing values. Additionally, timestamp data was also removed despite not explicitly being of an incompatible type, as the model should not learn the fixed timing of an attack in the past as a feature to predict future attacks which cannot take place in the past. Removing these 0 features has no impact explanations employing Shapley values due to the property of the null player. This process reduces the number of traffic flow features from 88 to 69. Following the pruning of features, data entries with invalid feature values were also removed. This ensures that the model is able to use gradient descent methods cleanly in the training process. Following the filtering of features and specific problematic data entries was the removal of the entire WebDDoS class and the respective data entries. This was advised by Sharafaldin et al. in Developing Realistic Distributed Denial of Service (DDoS) Attack Dataset and Taxonomy[2] explaining the creation of the dataset and expected use cases. The reason for the removal of WebDDoS was simply because of the lack of data entries. The final step in the filtering process is the removal of any duplicate data entries which may have differed in the features which were removed in the first step. The final steps of the data processing procedure were to normalize each feature via min-max normalization to ensure that all feature values lie between 0 and 1 to ease computation.

Attack Type	Raw Frequency	Post Processing Frequency	% Difference
BENIGN	113828	107230	-5.796
DNS	5071011	4773092	-5.874
LDAP	4095052	3876860	-5.328
MSSQL	10309945	9971677	-3.281
NetBIOS	7750776	7087440	-8.558
NTP	1202642	1195569	-0.588
SNMP	5159870	5037610	-2.369
SSDP	2610611	2568538	-1.612
UDP	7001800	6884146	-1.680
Syn	6473789	5496320	-15.10
TFTP	20082580	17859895	-11.07
UDP-lag	368334	331943	-9.880
Portmap	186960	177194	-5.224
WebDDoS	439	0	-100.0
Total	70427637	65367514	-7.185

Table 3.1: Processed Attack Frequencies

The final step in processing the dataset was balancing the classes to ensure that each label receives equal representation in training, as the frequency of attacks collected may not necessarily reflect actual frequency of attacks in model deployment. This is a possibility since the dataset was synthetically generated and likely holds some biases from the researchers from Sharafaldin et al.[2] The method of balancing the classes was to use reservoir sampling on all classes which possessed a greater number of data entries than the smallest class. In this case, the Benign class of data flow packets was the smallest at 107,230. Thus the other classes were randomly sampled without replacement for 107,230 entries at training time with a random seed. The random seed selected was 774.

Attack Type	Training Frequency
BENIGN	107230
DNS	107230
LDAP	107230
MSSQL	107230
NetBIOS	107230
NTP	107230
SNMP	107230
SSDP	107230
UDP	107230
Syn	107230
TFTP	107230
UDP-lag	107230
Portmap	107230
Total	1393990

Table 3.2: Attack Frequencies after Reservoir Sampling from Processed Frequencies

3.5 IDS Design

The machine learning architecture of choice was the CNN as specified in prior Section 2.2 Intrusion Detection Systems, for their ability to isolate features in tabular sequence data. For the experiment, two models will be developed and tested, a binary classification model sorting benign and malicious packets for anomaly detection, and a multiclass classification model sorting benign and specific attack types for signature detection.

3.5.1 Anomaly Detection IDS

The anomaly detection model comprises of 2 convolutional layers, a max pooling layer, and 2 fully connected neuron layers. The model is meant to take in a single channel, 69 feature long input which it filters on using a kernel of size 3 with a stride of 1. The output contains 64 channels and a Rectified Linear Unit (ReLU) activation function to introduce non-linearity. The following layer performs the same operation. The final layer before the fully connected neural net is a max pooling layer which groups 2 features and selects the max from each pool. These layers extract underlying features from the tabular data. The output is then passed through a ReLU function once more and then a dropout layer is applied. The dropout layer eliminates neuron signals with a probability of 0.5 for each neuron. This is to ensure that the model does not over fit to the training data, such that the model cannot rely on single neuron paths for given predictions. The following fully connected dense layers are of sizes 2048 and 128 with a final output being a 2 dimensional vector. These layers also utilise ReLU for computational efficiency, with the final prediction being passed through a sigmoid function to get probability logits for each predicted class. The binary cross entropy function then takes in the logit outputs to calculate the loss for each set of predictions.

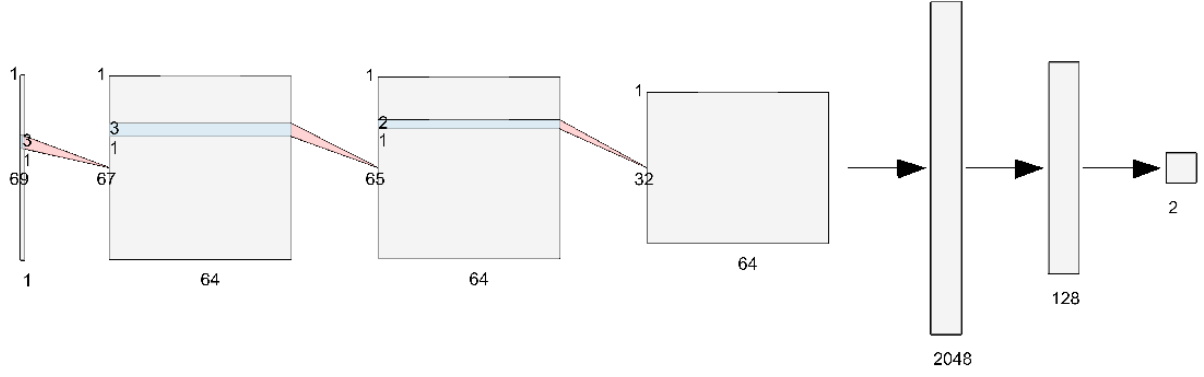


Figure 3.3: Anomaly Detection Model Architecture

The optimizer selected was the Adam optimizer [11] with a learning rate of 10^{-4} , implemented with weight decay of 10^{-6} . [12] Models were trained with a batch size of 256, meaning 256 predictions were averaged before an update was made to the model in order to smooth out the gradient descent steps.

3.5.2 Signature Detection IDS

The signature detection model architecture is constructed based on the anomaly detection model. The major differences include an additional convolutional layer which has a larger filter size of size 5 instead of 3, and the addition of an extra fully connected dense layer for the purposes of increasing the function space of the signature detection model. This change allows the signature detection model additional parameters for differentiating features between a greater number of possible outcomes. The fully connected neuron layers also change dimensionally as a result of the number of extracted features from the convolutional layers being different due to the additional convolution operation of the third layer. The resultant fully connected neuron layer sizes become 1920, 256, and 128 to produce a vector of 13 elements. The sigmoid function here is replaced with its n-dimensional equivalent, the softmax function to create the probability logits. The loss function of choice is the cross entropy function, the same generalized form of binary cross entropy used for the anomaly detection case.

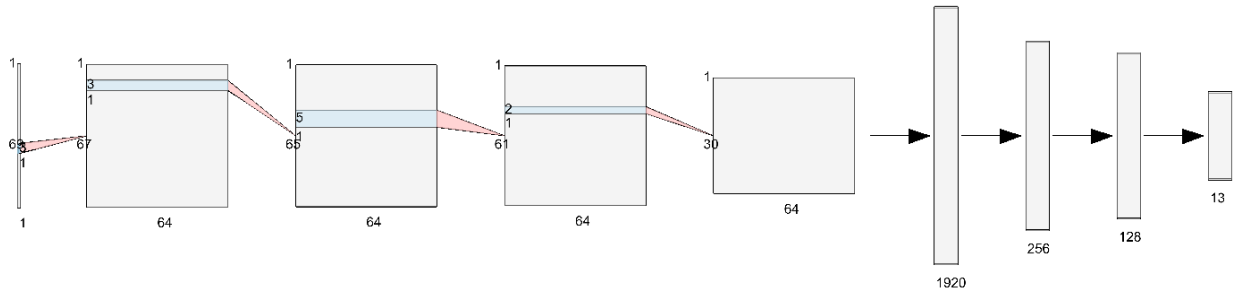


Figure 3.4: Signature Detection Model Architecture

3.6 Developing Explanations Innately

To explain the models without the use of post hoc methods, it will be necessary to alter the training process. The logical step would be to influence how the model updates and learns to nudge it towards using explainable features while not sacrificing too much performance. This can be accomplished by integrating an explanation method such as SHAP with the loss function before the back propagation weight update to model parameters. The primary road block to this method is that the SHAP values alone do not have any significance in a vacuum for updating the model to be more interpretable, but must be compared to a ground truth in order to provide a meaningful update to the model parameters accompanying the loss. In order to establish a ground truth, it is necessary to select a set of features for each label believed to be the most influential for selection of the label. These feature-label relations are best represented by knowledge graphs as a method of altering model training, pioneered by N. Díaz Rodríguez et al. in EXplainable Neural-Symbolic Learning (X-NeSyL) methodology to fuse deep learning representations with expert knowledge graphs: the MonuMAI cultural heritage use case.[9]

3.6.1 Knowledge Graphs

Knowledge graphs (KG) are a representation of relational understanding between a given concept and its subsequent components or related features.[9], [13] These knowledge graphs are inherently bipartite in nature given the Shapley values which knowledge graphs serve as a ground truth for are do not permit direct feature-feature relations or label-label relations.

For anomaly detection, the binary knowledge graph consists of two label nodes and each node's respective feature nodes. In the binary case of benign and malicious packet flow features, the features which indicate one class do not indicate another, providing a disconnected graph structure. For benign packets the features indicating a consistent flow of packets in the forward and backward direction are required, to indicate proper communication between a server and client in the network.

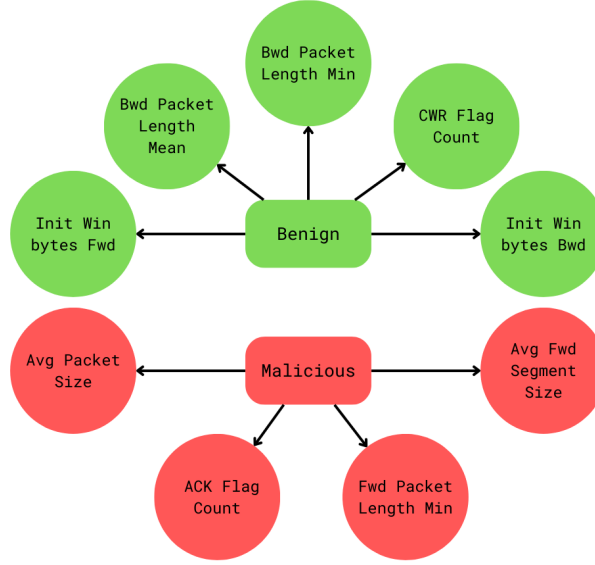


Figure 3.5: Binary Classification Knowledge Graph

Benign Feature	Reason for Selection
Init Win bytes Fwd	The quantity of information sent by the source to the destination within the first window captured by the Flowmeter tool. Important for gauging the forward flow of information. An extremely high value without an equal response in the Init Win bytes Bwd feature likely indicates malicious packet flow.
Bwd Packet Length Mean	The average length of data packets sent by the destination to the source in response. A high to moderate value indicates that the destination server is capable of responding, indicating benign and expected communication.
Bwd Packet Length Min	The minimum length of data packets sent by the destination to the source in response. A non-zero value indicates that the destination server is capable of responding, indicating benign and expected communication.
CWR Flag Count	Congestion Window Reduce Flag count for packets using TCP protocol. This flag issued by the sender upon receiving an explicit congestion experienced (ECE) flag from the receiver to indicate a reduction in transmission rate to reduce congestion. Communication between the source and destination to improvement to bandwidth indicates benign packet flow.
Init Win bytes Bwd	The quantity of information sent by the destination to the source as a response within the first window captured by the Flowmeter tool. Indicates a timely response from the server to the client, increasing likelihood of benign packet flow.

Table 3.3: Anomaly Detection Knowledge Graph Benign Features

Malicious Feature	Reason for Selection
Avg Packet Size	Mean packet size in the flow between source and destination. A large value indicates increased bytes/s being sent and such that communication channel bandwidth cannot sustain proper messaging between source and destination. High likelihood of malicious packet flow.
ACK Flag Count	Number of TCP Packets with ACK flag set to true. For exploitation type attacks, such as Syn floods, packets with SYN and ACK flags are sent from the destination in response. A large volume of ACK flags can indicate a high likelihood of malicious packet flow.
Fwd Packet Length Min	The minimum packet length in from the source to the destination in packet flow. If the minimum length of the packets is very large, it may indicate an attempt to overload the communication bandwidth from an reflection amplification attack.
Avg Fwd Segment Size	The average TCP segment size in the forward direction from the source to the destination. If the segment size is unusually large and close to the maximum it is likely a malicious reflection amplification attack trying to use up as much bandwidth as possible.

Table 3.4: Anomaly Detection Knowledge Graph Malicious Features

Assembling a knowledge graph for signature detection is significantly more difficult given the considerable increase in labels which each require their own set of core features. Given the choice of 69 total features after data processing picking features directly for a larger number of labels becomes a difficult task. To ease the task of hand picking features, a ranking of the top k features per each class was implemented first as a filtering method. Choosing $k = 5$ allowed for a proper reduction of features to consider. However, given the number of overlapping features important to each label when using a ranking method, the resultant knowledge graph requires further pruning to be usable.

The method of using a knowledge graph edit distance as an explanation metric with Shapley values will likely perform better if labels possess unique core features. This is based on the following section’s 3.6.2 implementation of the SHAP Loss method which encourages dependence on certain features for predicting specific labels based on Shapley values. If multiple labels are encouraged to increase dependence on the same small set of features thought to be most interpretable, there will likely be a greater amount of confusion between said predicted labels and subsequently a greater decrease in performance. Thus additional post processing was performed on the top k knowledge graph to create a pruned knowledge graph with significantly reduced overlap between feature and label pairs. It is impossible to remove all overlap between feature and label pairs given the setup and the processing of the data, however, the total reduction in overlapping features is considerable enough that the majority of labels do not rely on the same feature.

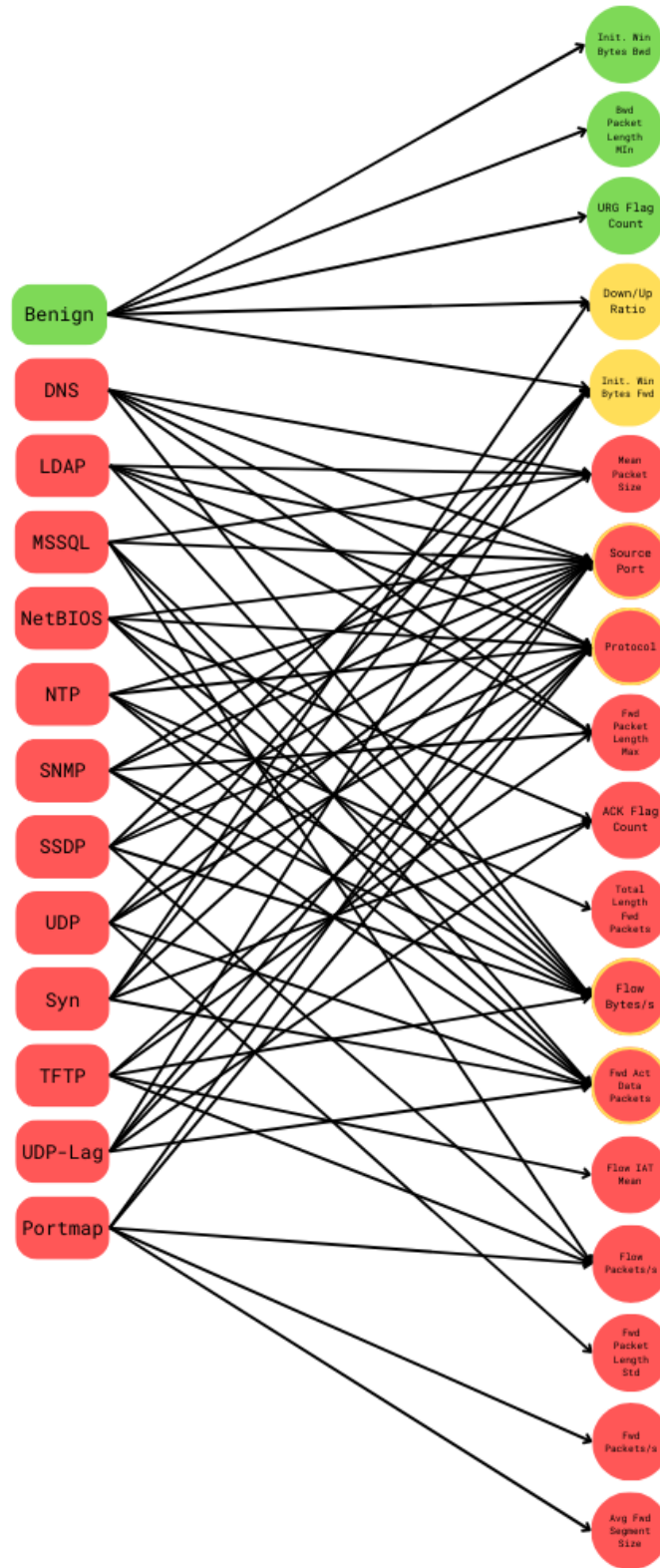


Figure 3.6: Topk Signature Detection Knowledge Graph, $k = 5$

Figure 3.7: Pruned Signature Detection Knowledge Graph



Table 3.5: Signature Detection Knowledge Graph Features

Label(s)	Feature	Reason for Selection
Benign	Init Win Bytes Bwd	See Table 3.3 for reasoning
Benign	Bwd Packet Length Min	See Table 3.3 for reasoning
Benign	Down/Up Ratio	Download/upload ratio relative to source of flow. A balanced number of packets in both directions indicate benign packet flow. A DDoS attack would see a skewed ratio of increased uploads from the source, decreasing the ratio.

Benign, Syn	Init Win Bytes Fwd	See Table 3.3 for benign feature reasoning. Syn floods also use this feature, as a high number of initial syn requests occur to exhaust a server’s resources and available ports in syn flood exploitation attacks. If a high initial volume does not occur, then a timeout will clear the ports listening for SYN/ACK responses, causing the attack to fail, thus this feature is required for Syn floods.
Syn, UDP- Lag	ACK Flag Count	Syn flood attacks prompt target servers to send out packets with SYN/ACK flags activated for continuing the TCP handshake, thus a high number of ACK flags may indicate a Syn flood. The analysis of Sharafaldin et al. when developing the CICDDoS2019 dataset[2] shows that ACK Flag Count is the best feature for determining UDP-Lag. This is likely a product of the lag-switch hardware or software used to simulate this attack. This is counter intuitive to standard UDP based protocol exploitation attacks which do not exhibit TCP ACK flags.
NTP	Fwd Act Data Packets	NTP reflection amplification attacks are a UDP based attack which has a greater amplification factor than all other attacks in the dataset. This results in a significantly larger packet count with non-zero payloads for time syncing across the network. Thus, the Fwd Act Data Packets feature is a good choice, as it measures the number of packets with at least 1 byte of payload (non-header) data.
DNS, MSSQL, SNMP, SSDP, UDP, Portmap	Source Port	The reflection attack types here use specific source ports on legitimate servers used as reflectors and are not at risk of having their IP addresses and ports spoofed. This makes the source port a reliable indicator. The DNS source port is often port 53 for TCP and UDP. The MSSQL source ports are 1433, 4022, 135, 1434, and UDP 1434. The source port of SNMP is UDP 161 and 162. The source port of SSDP is UDP 1900. Portscan and UDP do not have any specified ports, however, these attacks have the benefit of choosing many of the unregistered higher number ports, which the other attacks do not often use.
Portmap	Flow Pack- ets/s	The number of packets in the flow per second is indicative of a portscan attack, as there would be a sharp increase in packets with no payload (differentiating from NTP which has payload) to scan a wide range of ports on the target system or network. A portscan involves attempting to connect to various ports on the target to identify which ones are open and listening for incoming connections.

Portmap	Fwd Packets/s	See reason above. Follows a similar line of thought, but this feature specifically isolates for the packets from the source and none of the response packets from the destination for the portscan.
SNMP, LDAP, TFTP	Flow Bytes/s	SNMP, LDAP, and TFTP are all data management protocols, and thus a DDoS attack perpetrated on these protocols would likely have a large byte signature instead of an extensively large packet signature, like an NTP reflection amplification.
NetBIOS, TFTP	Protocol	NetBIOS DDoS attacks can use UDP or TCP protocol. NetBIOS Name Service (NBNS) typically uses UDP (User Datagram Protocol), while NetBIOS Session Service (SMB) uses TCP (Transmission Control Protocol), and this feature can help differentiate between these attack subtypes. TFTP uses the UDP transport protocol, which has a protocol ID of 17 as opposed to the protocol ID of TCP being 6. This indicates that TFTP will have a higher mean protocol ID value.

3.6.2 SHAP Loss

The goal of SHAP Loss is to influence the loss function of the detection model during training to encourage reliance on features with human interpretable reasoning. In order to accomplish this, the SHAP Loss function needs to compare how important a feature’s influence is on a given prediction. The Shapley Value must then be compared to a ground truth value of feature importance to return a value which increases the loss when the Shapley Value does not match that of the ground truth.

The implementation for the SHAP Loss algorithm is a modified form used by N. Díaz Rodríguez et al.[9] as follows:

Algorithm 1 Applies to the loss function the polarity difference between sampled SHAP values, and the ground truth KG

```

procedure  $Loss_{SHAP}(L, \hat{y}, \phi_{\hat{y}}, KG, h)$ 
  while  $l < num\_labels$  do
     $\phi_l = \sum \phi_{\hat{y}}, \text{ where } \hat{y} = l$ 
  end while
   $\phi = [\phi_0, \phi_1, \dots, \phi_l]$ 
   $\beta = \sum (-KG \cdot \phi)^+$  ▷ Keep only KG misaligned SHAP attribution
  return  $L * (h\beta + 1)$ 
end procedure

```

The first while loop aggregates the Shapley values of all predictions for each label and feature to create a matrix of labels by features. This matrix of Shapley values for specific feature-label pairs is then dotted with the inverse of the ground truth knowledge graph adjacency matrix. This provides us with a positive value when a the Shapley Value attribution

for a feature and label disagree in sign with the knowledge graph edge (represented by a 1, and a -1 for the lack of an edge between nodes). Only positive values of the matrix are kept, summed, and scalar multiplied by a hyper parameter h before being multiplied with the baseline loss function. Thus, if there is a large number of misattributed features, the resultant SHAP Loss function will increase significantly, and if there are a significant quantity of correctly attributed features, the SHAP Loss function is equal to or considerably near the original baseline loss function. Additionally, it may not be necessary to pass in all predictions of a batch when calculating SHAP Loss, as a fraction of a sufficiently large batch size may still provide good approximations for Shapley values when using the DEEPLIFT algorithm[8].

3.6.3 SHAP Graph Edit Distance

The SHAP Graph Edit Distance (SHAP GED) utilises a threshold parameter and Shapley values to generate a knowledge graph to evaluate against the ground truth knowledge graph. If aggregate Shapley values for a given correctly predicted label and feature are above a certain threshold, it indicates the model’s understanding of a strong relationship between the feature and label, thus an edge can be assigned to the pair of nodes in the knowledge graph. This process can be repeated for each pair of feature-label nodes in parallel. The algorithm for SHAP GED aggregates the Shapley values for the correctly predicted data samples on their labels and features, which forms a matrix of labels and features. The threshold is then applied, and all values above the threshold are masked with a 1 and all values below are masked with a -1. This process generates a knowledge graph representation as an adjacency matrix similar to the ground truth knowledge graph supplied. It is then possible to sum over all inconsistencies between the Shapley Value generated knowledge graph and the ground truth knowledge graph to arrive at a difference metric between the two interpretations of data features.

Algorithm 2 Finds the edge difference between a ground truth KG and a SHAP generated KG

```

procedure  $GED_{SHAP}(L, y, \hat{y}, \phi_y, KG, t)$ 
  while  $l < num\_labels$  do
     $\phi_l = \sum \phi_{\hat{y}}$ , where  $\hat{y} = l = y$ 
     $\phi_l = \frac{\phi_l}{|max(\phi_l)|}$  ▷ Normalize feature attributions per class
  end while
   $\phi = [\phi_0, \phi_1, ..., \phi_l]$ 
   $\phi = -1$ , where  $\phi < t$  ▷ Mask Shap values
   $\phi = 1$ , where  $\phi \neq -1$ 
  return  $\sum 1$ , where  $KG \neq \phi$  ▷ simplified GED
end procedure

```

This algorithm uses a modified version of N. Díaz Rodríguez et al.’s implementation which uses a traditional GED between the Shapley Value generated knowledge graph and the ground truth knowledge graph.[9] The method implemented by N. Díaz Rodríguez et al.

is significantly more punishing on Shapley Value feature misattributions, as the traditional implementation of a graph data structure does not define all possible nodes as part of the graph, and thus additional graph edits must be made to insert the nodes, whereas the algorithm used in this thesis only checks for the difference of edges and assumes that all nodes exist and that the graphs are disconnected in nature. This modification was made in attempt to reduce computational complexity, as it is more simple to check for differences in graph edges via an adjacency matrix than it is to step through a graph data structure to arrive at a proper edit distance metric. The modification achieves the same purpose albeit with results differing in magnitude as a trade-off for computational efficiency.

Chapter 4

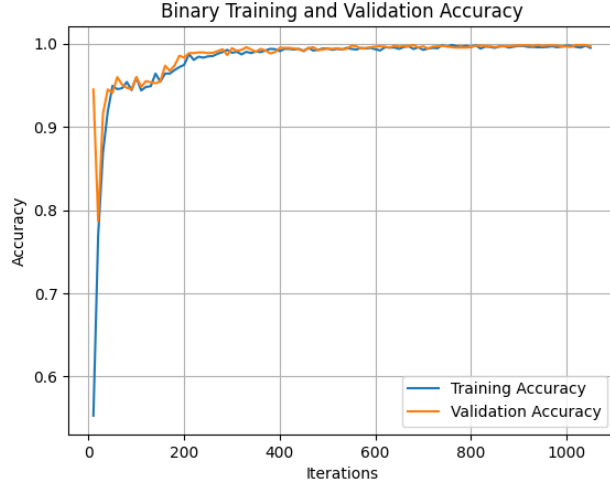
Experiment Takeaways

4.1 Model Performance

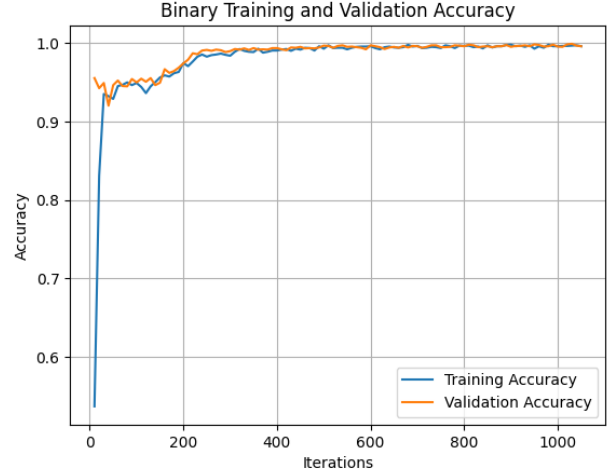
As was previously noted in Experiment Objectives 3.1, the metric used to evaluate model performance would be accuracy. This metric was chosen on the basis of IDS requiring a low false positive rate to maintain proper system integrity and low false negative rate to uphold system availability to regular users. Based on prior works, an accuracy of 99% can be achieved with CNN architectures for the binary anomaly detection case. [1], [4], [5] Based on inconsistent results for the signature detection case, no hard objective will be set for overall accuracy, and criterion of a higher accuracy would be better. It is possible however, to reduce a signature detection model to that of an anomaly detection model, by only evaluating the accuracy of the benign class against all other attack classes. Thus, for the signature detection model, the objective will include hard objective of a 99% accuracy on classifying benign packets and a general criteria for all classes with a higher accuracy being better.

4.1.1 Anomaly Detection Performance

The results from anomaly detection models were taken after 1050 iterations with a batch size of 256. The training and validation accuracies recorded in figures 4.1a, 4.1b during the training session quickly meet the objective as expected based trends from training and validation accuracies recorded in prior works. [4], [5] This shows that the model is able to quickly pick up on the features which differentiate benign packet flows from malicious packet flows.



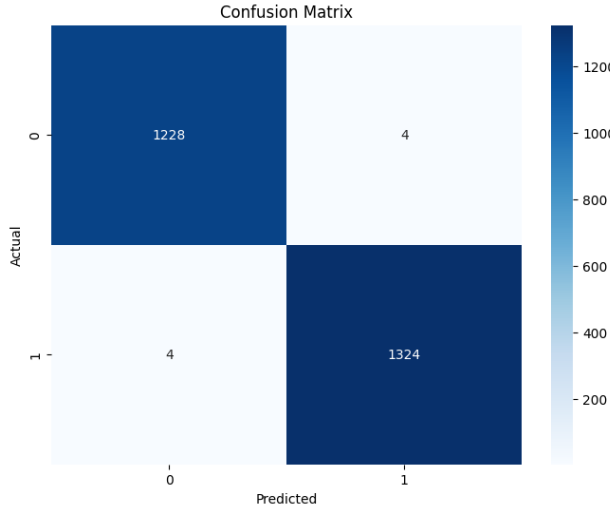
(a) Anomaly Detection Baseline



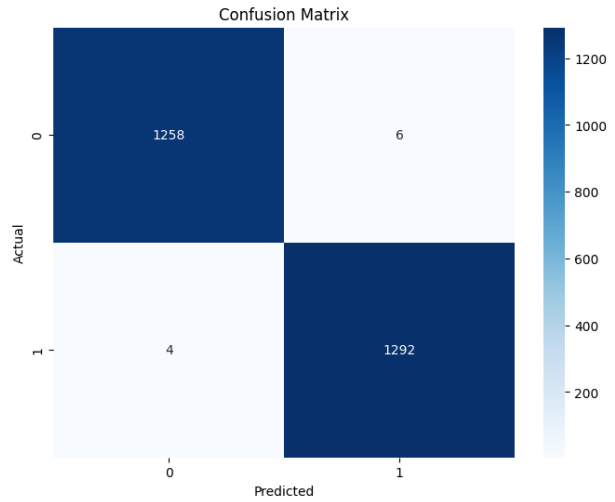
(b) Anomaly Detection SHAP Loss $h = 1$

Figure 4.1: Anomaly Detection Model Accuracies

The following confusion matrices show the number of predictions for each class against the ground truth labels for the predictions, and subsequently match the accuracies from the figures above.



(a) Anomaly Detection Baseline



(b) Anomaly Detection SHAP Loss $h = 1$

Figure 4.2: Anomaly Detection Model Confusion Matrices @ 1050 Iterations

4.1.2 Signature Detection Performance

The results from signature detection models were taken after 500 epochs of 5446 iterations and 100 epochs of 5446 iterations with a batch size of 256. The training and validation accuracies recorded in figures 4.3a, 4.3b follow the trends of the anomaly detection models,

however they plateau earlier at 0.73 for the baseline signature detection model, and 0.65 for the SHAP Loss signature detection model respectively.

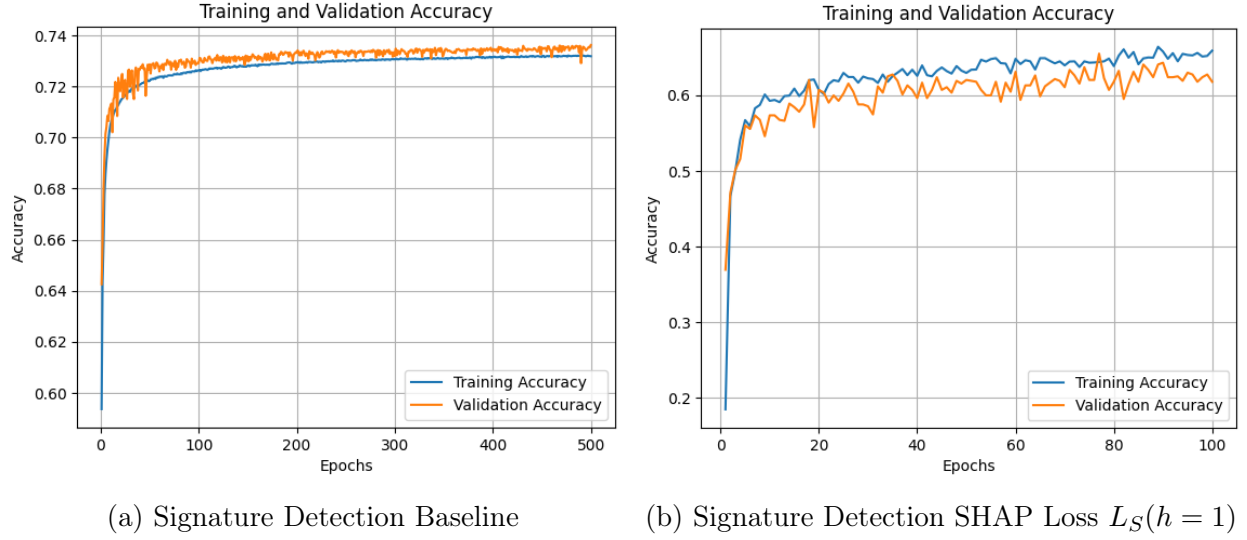


Figure 4.3: Signature Detection Model Accuracies

This shows that there are likely some classes which are difficult to classify based on the provided features, or the models did not correctly learn the features required to differentiate between specific classes. This may be particularly problematic for the ad hoc training explanation method of comparing knowledge graphs, as the baseline model demonstrates that it is not able to always correctly classify specific class labels.

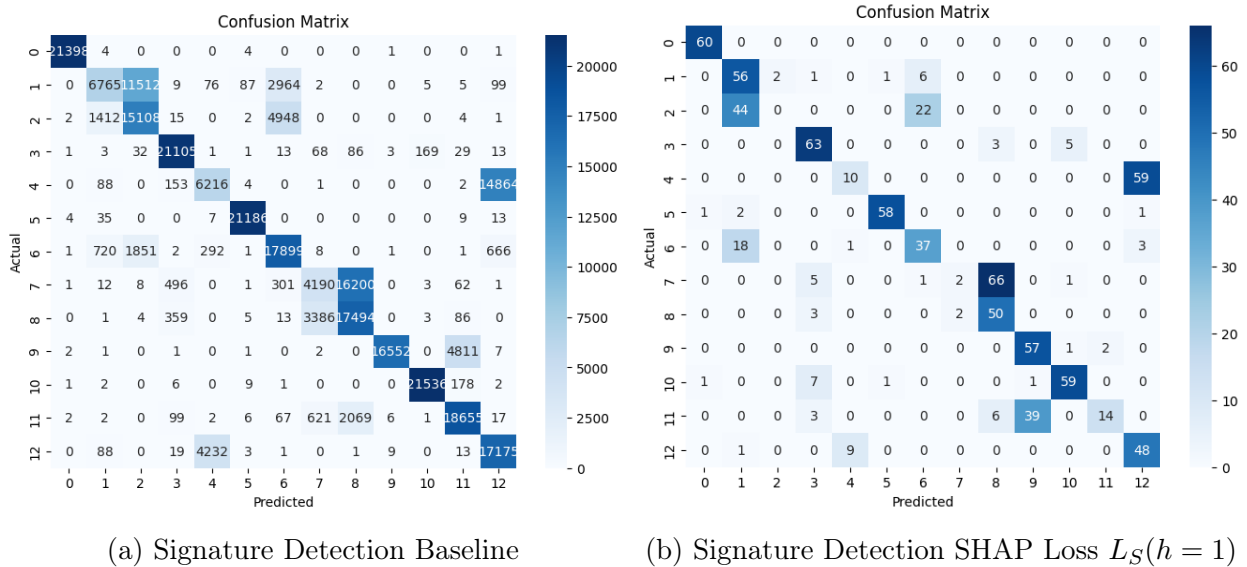


Figure 4.4: Signature Detection Model Confusion Matrices @ 500 Epochs and 100 Epochs

Despite this issue, the benign class remains unaffected, and retains a high level of accuracy when the model is reduced to the binary anomaly prediction case. This can be seen in the

confusion matrices above in figures 4.4a, 4.4b. The predicted class for 0 (benign class) retains a high accuracy against the ground truth actual labels, and this is visible in training when grouping together the predictions of all subsequent classes into a malicious super-class, tracked in the following figure 4.5a, 4.5b.

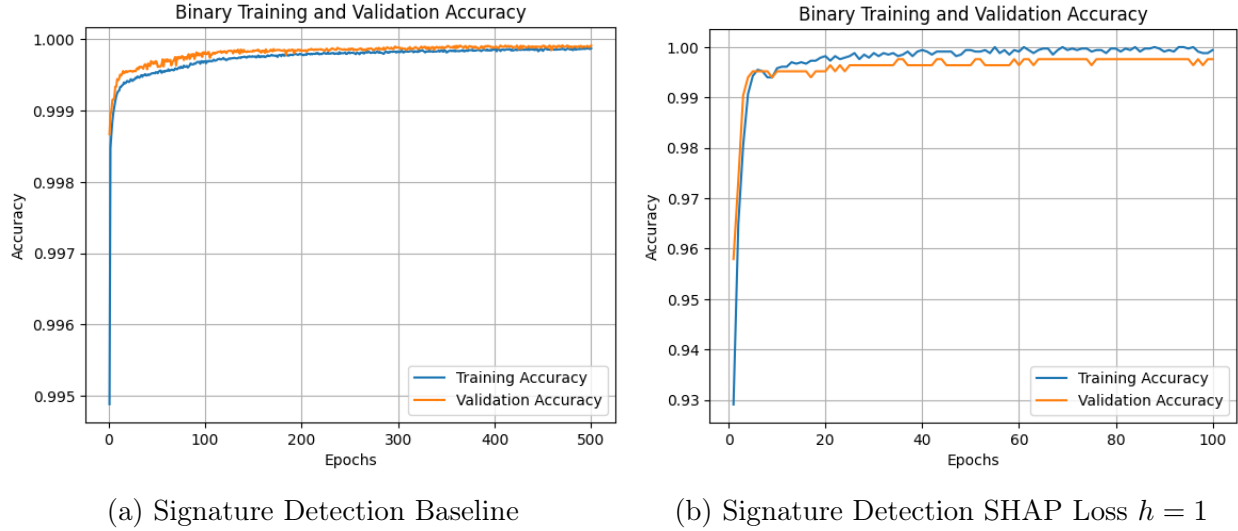


Figure 4.5: Signature Detection Model Benign Class Accuracies

This shows that when the signature detection model is reduced to the anomaly detection model, it is still able to uphold an accuracy of 99% to meet the expected accuracies from prior works. This likely indicates that the selection of benign packet flow features in the ground truth knowledge graph has a high likelihood of matching the model’s own learned feature importance.

4.2 Model Explanations

The proposed metrics for ad hoc explanations covered in Section 3.6.3 are examined and discussed here, along with the subsequent post hoc white box explanation to affirm the results obtained from SHAP Loss and SHAPGED.

4.2.1 Anomaly Detection Explanation

The modified SHAPGED measurement proposed in Section 3.6.3 is calculated at the conclusion of each iteration of the training process. Each iteration of batch size 256 had the SHAP Loss calculated on a sample of 100 randomly selected inputs and predictions in the batch to approximate the Shapley values of the entire batch. Based on the usage of the DEEPLIFT algorithm meant to approximate Shapley values, a sample size of 100 provides a good estimate for the global explanation of the model. [14]

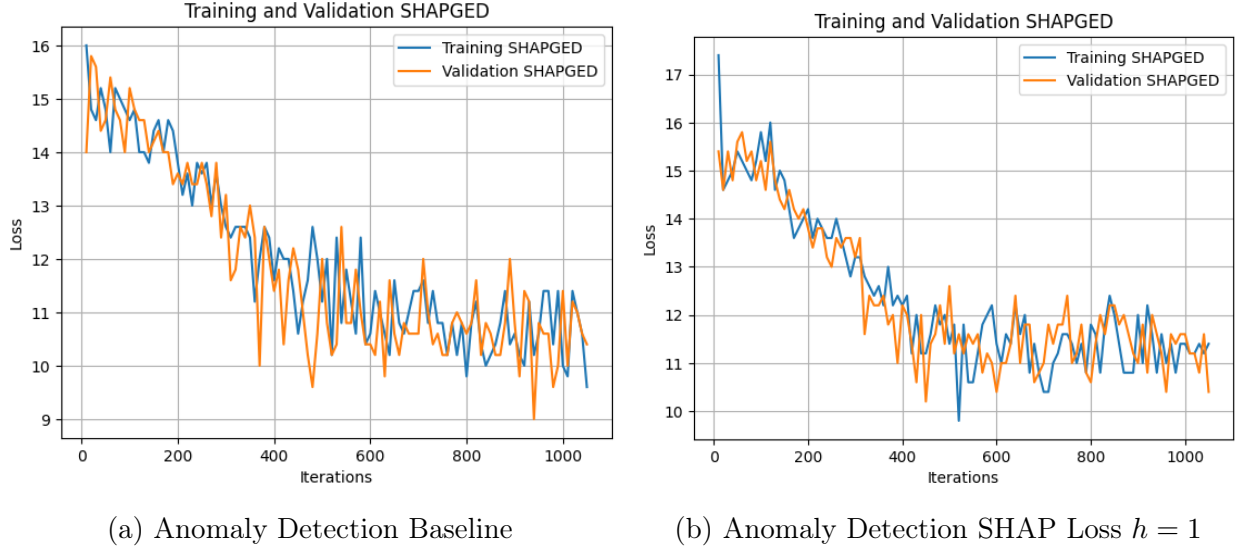


Figure 4.6: Anomaly Detection Model SHAP Graph Edit Distances in Training

For anomaly detection, the model trained with SHAP Loss exhibits very similar SHAPGED to the baseline model. Combined with the Shapley Value analysis in the following section, it is apparent that the important features selected in the anomaly detection knowledge graph 3.5 match very closely with the feature importance learned by the baseline anomaly detection model without any influence from SHAP. Thus, the similarity between the results of the SHAPGED metric is the expected outcome given the similarity of feature choice between the model and the researcher assigned knowledge graph. The affirmation of an identity case when using SHAP Loss and SHAPGED proves that if the model is already explainable, then the use of SHAP Loss and SHAPGED will not hinder the innate interpretability and that the measure of interpretability provided by SHAPGED during training is accurate to a degree which is human-interpretable.

As demonstrated by the Shapley values in the subsequent figures 4.7a, 4.7a, the post hoc analysis shows the similarities between the Shapley Value explanation on the baseline model, and the SHAP Loss model influenced by the researcher ground truth knowledge graph 3.5. The observable similarities between the features and their sorted placement on the chart indicate their importance. The observable important features learned by the baseline model for predicting the Benign class are:

- Bwd Packet Length Min
- CWE Flag Count (CWR Flag Count)
- URG Flag Count
- Init Win Bytes Fwd
- Init Win Bytes Bwd

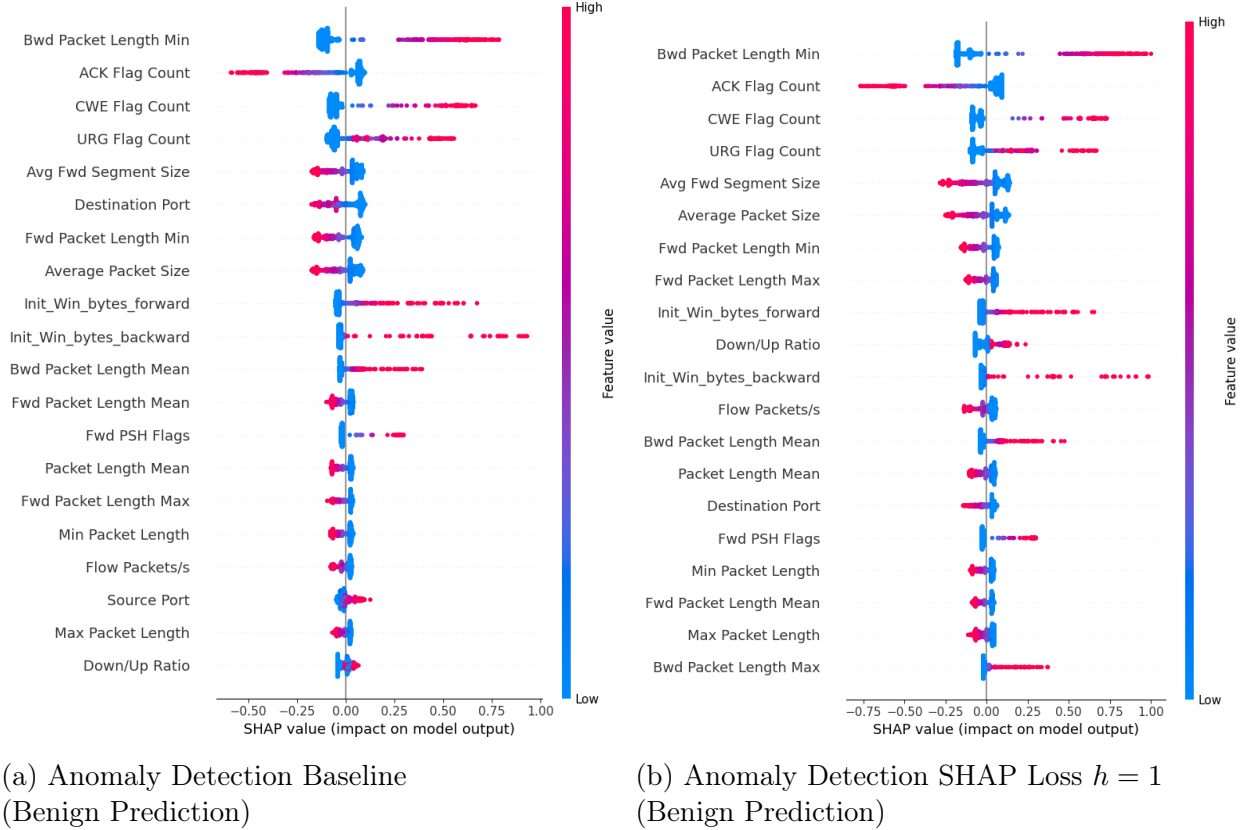


Figure 4.7: Anomaly Detection Model Shapley Values for Benign Predictions

This shows a significant overlap in 4 of 5 features represented in the knowledge graph. Despite Bwd Packet Length Mean’s exclusion from the most important features based on Shapley values of the baseline model, it is important to note that Bwd Packet Length Mean is highly correlated to Bwd Packet Length Min and can be interpreted as an important feature of the ground truth knowledge graph which may indicate benign packets. It is still valid to conclude that Bwd Packet Length Mean is indicative of the benign class given its significant Shapley value, albeit in relation to the most important feature Bwd Packet Length Min. Again, the similarities between these metrics supports the similarities of the prior SHAP GED measurements.

4.2.2 Signature Detection Explanation

The signature detection case of the SHAP Loss model displays significantly more deviation from the Baseline model when compared to the anomaly detection case. It is also important to note that the figure 4.8a is the subsequent SHAPGED for the pretrained weights of the baseline model after 500 epochs. This was because the SHAPGED metric was not yet fully realized and implemented at the time of training the baseline multiclass model. Thus, the pretrained model weights of the baseline model after 500 epochs was then loaded and trained without SHAP Loss to record the SHAPGED.

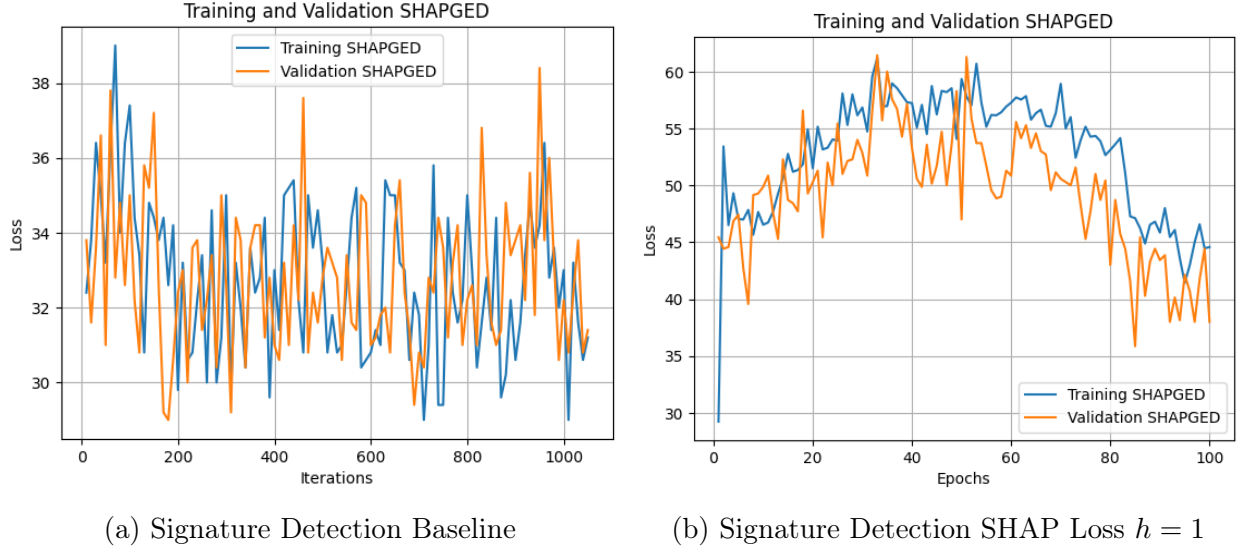


Figure 4.8: Signature Detection Model SHAP Graph Edit Distances in Training

There is no clear trend in improving the explainability via a decrease in SHAPGED, however the SHAPGED has reached a low point of 32, below all recorded values of the SHAP Loss signature detection model. This is a likely a product of the pretrained weights having received more updates than the SHAP Loss model without pretraining, which only trained for 100 epochs.

For the signature detection SHAP Loss model, instead of observing a decreasing SHAPGED metric consistently throughout training, the model instead trade the minimization of the explainability metric for the optimization of performance in the first 50 epochs. Once a plateau has been reached in performance, the model then subsequently chooses to optimize for explainability independently of performance. This trade-off between performance and explainability likely indicates that each label's set of important features from the ground truth knowledge graph does not provide enough information alone to properly describe and differentiate between labels. Additional experimentation is required to properly conclude this hypothesis.

Examining the Shapley values of labels for signature detection, it is clear that the SHAP Loss was able to shift the benign class label's dependence on features signaling the negative presence of other class labels to a greater dependence on its own indicator features. The primary example of this can be observed in the benign class' Shapley values in figure 4.9a, 4.9b. In figure 4.9a, the baseline signature detection case, we can observe that the most important features for determining the benign packet flow class are Flow Packets/s, Flow Bytes/s, Source Port, and Protocol. These features are not indicative of the benign class, but reasonably important for categorizing between the malicious classes, and thus these features are seen instead to negatively contribute to the classification of benign classes.

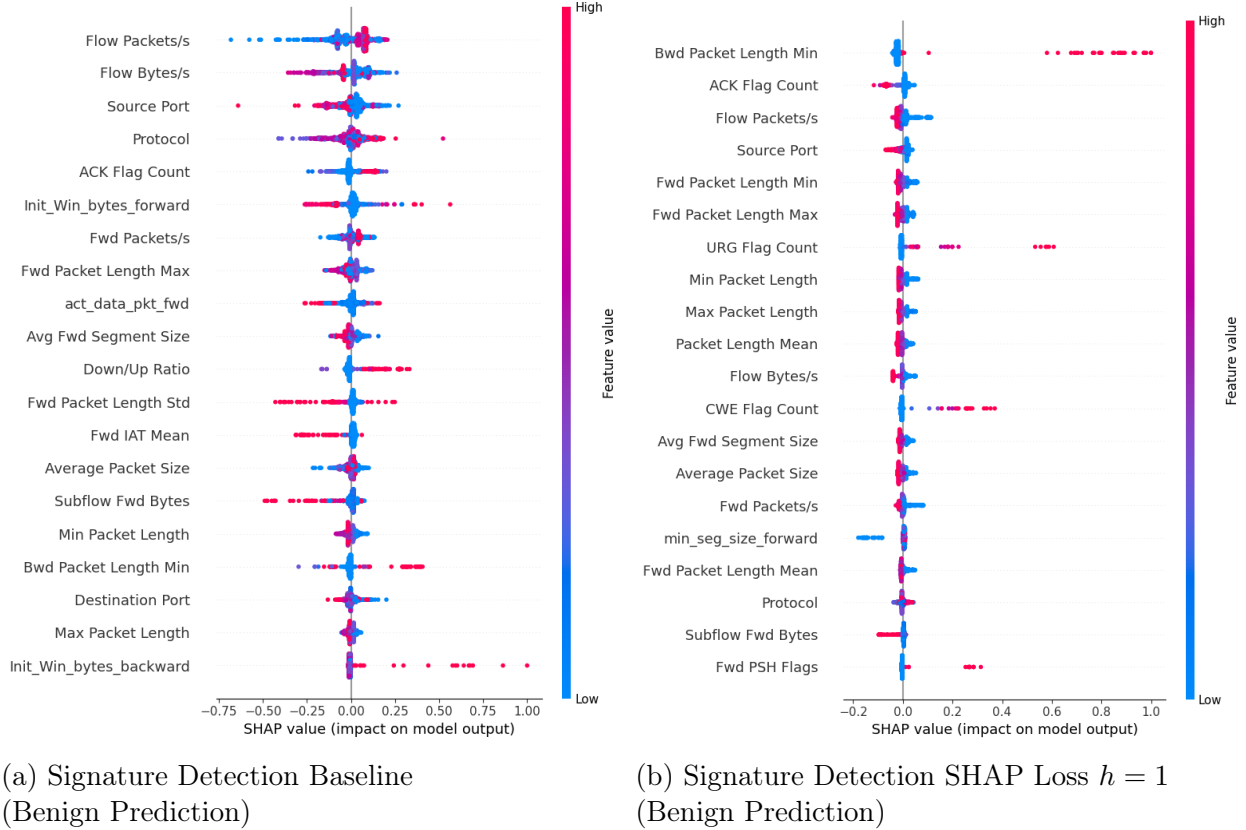


Figure 4.9: Signature Detection Model Shapley Values for Benign Predictions

Although this makes sense to learn for classification with a fixed number of classes, it is not a good implementation for deployment, as it does not accurately learn what features a benign packet flow exhibits, but rather what the other malicious packet flows do not. Through the use of SHAP Loss, experimental evidence shows that it is possible to recover the original set of features which indicate benign packet flow, as seen in figure 4.9b. The figure illustrates the most important features the model learns for classifying benign packets being Bwd Packet Length Min, URG Flag Count, and CWE (CWR) Flag Count, which matches the set of features for benign classification in the anomaly detection case.

These results are promising, as it demonstrates SHAP Loss' ability to increase the interpretability of multi-class predictions, despite a few short comings. A primary concern for the method of using SHAP Loss to increase interpretability is when multiple labels share important features. Based on the confusion matrix in figure 4.4a, where a noticeable increase in misclassifications happens between classes which share significant overlap in important features supplied by the ground truth knowledge graph.

This is a significant weakness of the SHAP Loss and SHAPGED method, as it shows the limits of interpretability based on the fidelity of features. This constraint is introduced by the knowledge graph used to calculate SHAP Loss and SHAPGED, as the knowledge graph cannot consider the value of each feature in addition to its influence. This limitation implies that SHAP Loss and SHAPGED are only capable of influencing which features the model should associate certain labels with, but is unable to associate specific ranges of feature

values with labels.

Potential solutions to this limitation that have yet to be tested include introducing weighted edges to the knowledge graph, binning ID based features into separate features to increase the interpretability fidelity. By introducing weighted edges to the knowledge graph supplied to the model, and by allowing the model itself to create weighted edges from its training-calculated Shapley values, the issue of feature-value fidelity would be solved via the weighted edges changing to reflect how important a feature would be to a prediction conditioned on its value. Alternatively, by binning specific features into multiple features based on value, the knowledge graph would have increased fidelity via allowing for specific selections of values for features that would be vitally important for predicting specific classes.

Chapter 5

Closing Remarks

5.1 Final Notes

Despite this limitation of SHAP Loss and SHAPGED, the results of the experiments demonstrate promising outcomes for what is possible with additional data processing the weaknesses presented by the use of knowledge graphs. The anomaly detection model consistently achieved high accuracies, indicating its effectiveness in distinguishing between benign and malicious packet flows while maintaining strong explainability of features for predictions. Moreover, the signature detection model, although facing challenges in classification, showed resilience in maintaining high accuracy for benign packet classification when reduced to the binary anomaly prediction case. Additionally, the utilization of SHAP Loss and SHAPGED provided reasonable ad hoc insights into feature importance during training. This highlights the potential to enhance model interpretability while balancing performance based on researcher assumptions of the data. However, the method's limitations, particularly in cases of shared important features among multiple classes, suggest avenues for further exploration and refinement. Overall, the thesis findings underscore the inner workings of intrusion detection systems and the importance of continued research to improve their effectiveness and interpretability.

References

- [1] R. Younisse, A. Ahmad, and Q. Abu Al-Haija, “Explaining intrusion detection-based convolutional neural networks using shapley additive explanations (shap),” *Big Data and Cognitive Computing*, vol. 6, no. 4, 2022, ISSN: 2504-2289. DOI: [10.3390/bdcc6040126](https://doi.org/10.3390/bdcc6040126). URL: <https://www.mdpi.com/2504-2289/6/4/126>.
- [2] I. Sharafaldin, A. H. Lashkari, S. Hakak, and A. A. Ghorbani, “Developing realistic distributed denial of service (ddos) attack dataset and taxonomy,” in *2019 International Carnahan Conference on Security Technology (ICCST)*, 2019, pp. 1–8. DOI: [10.1109/CCST.2019.8888419](https://doi.org/10.1109/CCST.2019.8888419).
- [3] I. Sharafaldin, A. Gharib, A. Habibi Lashkari, and A. Ghorbani, “Towards a reliable intrusion detection benchmark dataset,” *Software Networking*, vol. 2017, pp. 177–200, Jan. 2017. DOI: [10.13052/jsn2445-9739.2017.009](https://doi.org/10.13052/jsn2445-9739.2017.009).
- [4] D. Akgun, S. Hizal, and U. Cavusoglu, “A new ddos attacks intrusion detection model based on deep learning for cybersecurity,” *Computers & Security*, vol. 118, p. 102748, 2022, ISSN: 0167-4048. DOI: <https://doi.org/10.1016/j.cose.2022.102748>. URL: <https://www.sciencedirect.com/science/article/pii/S0167404822001432>.
- [5] A. Meliboyev, J. Alikhanov, and W. Kim, “1d cnn based network intrusion detection with normalization on imbalanced data,” Feb. 2020, pp. 218–224. DOI: [10.1109/ICAIC48513.2020.9064976](https://doi.org/10.1109/ICAIC48513.2020.9064976).
- [6] K. Singh, A. Mahajan, and V. Mansotra, “1d-cnn based model for classification and analysis of network attacks,” *International Journal of Advanced Computer Science and Applications*, vol. 12, no. 11, 2021. DOI: [10.14569/IJACSA.2021.0121169](https://doi.org/10.14569/IJACSA.2021.0121169). URL: <http://dx.doi.org/10.14569/IJACSA.2021.0121169>.
- [7] L. S. Shapley, “A value for n-person games,” in *Contributions to the Theory of Games II*, H. W. Kuhn and A. W. Tucker, Eds., Princeton: Princeton University Press, 1953, pp. 307–317.
- [8] A. Shrikumar, P. Greenside, and A. Kundaje, “Learning important features through propagating activation differences,” *CoRR*, vol. abs/1704.02685, 2017. arXiv: [1704.02685](https://arxiv.org/abs/1704.02685). URL: <http://arxiv.org/abs/1704.02685>.
- [9] N. D. Rodríguez, A. Lamas, J. Sanchez, G. Franchi, I. Donadello, S. Tabik, D. Filliat, P. Cruz, R. Montes, and F. Herrera, “Explainable neural-symbolic learning (x-nesyl) methodology to fuse deep learning representations with expert knowledge graphs: The monumai cultural heritage use case,” *CoRR*, vol. abs/2104.11914, 2021. arXiv: [2104.11914](https://arxiv.org/abs/2104.11914). URL: <https://arxiv.org/abs/2104.11914>.

- [10] M. C. P. Saheb, M. S. Yadav, S. Babu, J. J. Pujari, and J. B. Maddala, “A review of ddos evaluation dataset: Cicddos2019 dataset,” in *Energy Systems, Drives and Automations*, J. R. Szymanski, C. K. Chanda, P. K. Mondal, and K. A. Khan, Eds., Singapore: Springer Nature Singapore, 2023, pp. 389–397, ISBN: 978-981-99-3691-5.
- [11] D. P. Kingma and J. Ba, *Adam: A method for stochastic optimization*, 2017. arXiv: [1412.6980 \[cs.LG\]](#).
- [12] M. Andriushchenko, F. D’Angelo, A. Varre, and N. Flammarion, *Why do we need weight decay in modern deep learning?* 2023. arXiv: [2310.04415 \[cs.LG\]](#).
- [13] A. S. d. Garcez, D. M. Gabbay, and K. B. Broda, *Neural-Symbolic Learning System: Foundations and Applications*. Berlin, Heidelberg: Springer-Verlag, 2002, ISBN: 1852335122.
- [14] S. M. Lundberg and S. Lee, “A unified approach to interpreting model predictions,” *CoRR*, vol. abs/1705.07874, 2017. arXiv: [1705.07874](#). URL: <http://arxiv.org/abs/1705.07874>.
- [15] M. T. Ribeiro, S. Singh, and C. Guestrin, “"why should I trust you?": Explaining the predictions of any classifier,” *CoRR*, vol. abs/1602.04938, 2016. arXiv: [1602.04938](#). URL: <http://arxiv.org/abs/1602.04938>.
- [16] A. Holzinger, A. Saranti, C. Molnar, P. Biecek, and W. Samek, “Explainable ai methods - a brief overview,” in *xxAI - Beyond Explainable AI: International Workshop, Held in Conjunction with ICML 2020, July 18, 2020, Vienna, Austria, Revised and Extended Papers*, A. Holzinger, R. Goebel, R. Fong, T. Moon, K.-R. Müller, and W. Samek, Eds. Cham: Springer International Publishing, 2022, pp. 13–38, ISBN: 978-3-031-04083-2. DOI: [10.1007/978-3-031-04083-2_2](#). URL: https://doi.org/10.1007/978-3-031-04083-2_2.

Appendix A

Code listing

A.1 Training Code

```
1 from pathlib import *
2 import json
3
4 import numpy as np
5 import pandas as pd
6 import torch
7 import torch.nn as nn
8 import torch.nn.functional as F
9 import torch.optim as optim
10 from torch.utils.data import Dataset, DataLoader, random_split
11 from tqdm import tqdm
12 import shap
13
14 from data.dataset import CICDDoS2019Dataset
15 from model import BaselineCNN
16
17 device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
18
19 def get_shap(model:torch.nn.Module, data:torch.Tensor, size:int=100):
20     ind = torch.randperm(data.size(0))
21     ind = ind[:size]
22     de = shap.DeepExplainer(model, data[ind])
23     shap_values = de.shap_values(data, check_additivity=False)
24     return shap_values.squeeze()
25
26 def shap_loss(shap_values:np.ndarray, KG:np.ndarray, pred:
27     torch.Tensor, h:int=1, exp:bool=False):
28     label_shap = []
29     for label in range(KG.shape[-1]):
```

```

30     ind = [j for j, x in enumerate(pred) if x == label]
31     local_shap = shap_values[ind,:,label]
32     label_shap.append(local_shap.sum(axis=0))
33
34     beta = np.multiply(-KG, np.stack(label_shap).T)
35     beta[beta < 0] = 0
36
37     if exp:
38         return np.exp(h*(np.sum(beta)))
39     return h*(np.sum(beta)) + 1
40
41 def get_sag(shap_values, pred, labels, sag_threshold:int=None, topk:
int=None):
42     sag = []
43     for class_label in range(shap_values.shape[-1]):
44         ind = [j for j, x in enumerate(pred) if x == class_label]
45         correct_ind = [i for i in ind if pred[i] == labels[i]]
46         local_shap = shap_values[correct_ind]
47         local_shap = local_shap.sum(axis=0)[: ,class_label]
48         local_shap /= abs(local_shap).max() + 1e-12
49         if topk:
50             ind = np.argsort(local_shap.flatten())[-topk:]
51             mask = np.zeros_like(local_shap, dtype=bool)
52             ind = np.unravel_index(ind, local_shap.shape)
53             mask[ind] = True
54             local_shap[~mask] = -1
55         else:
56             mask = local_shap < sag_threshold
57             local_shap[mask] = -1
58         sag.append(local_shap)
59     sag = np.stack(sag).T
60     sag = np.sign(sag)
61     return sag
62
63 def get_accuracy(predictions, labels):
64     predictions = torch.argmax(predictions,dim=1)
65     if predictions.shape != labels.shape:
66         raise ValueError(f"Predictions and labels must have the same
shape. predictions of size {predictions.shape} and labels
of size {labels.shape}")
67
68     correct = (predictions == labels).sum().item()
69
70     total = labels.numel()
71     accuracy = correct / total
72
73     return accuracy

```

```

74
75 def run_shap_training(model, dataset:Dataset, epochs:int, KG, split:
    list[float]=[0.8,0.2], batch_size:int=64, lr:int=0.001, checkpoint
    :int=None, log:int=None, h=1):
76     training_stats = {'train_pred': [], 'train_labels': [], '
        train_loss': [], 'train_SHAPGED': [], 'val_pred': [], '
        val_labels': [], 'val_loss': [], 'val_SHAPGED': []}
77
78     if seed:
79         np.random.seed(seed)
80     if len(split) == 2:
81         train_set, val_set = random_split(dataset, split)
82         test_set = None
83     elif len(split) == 3:
84         train_set, val_set, test_set = random_split(dataset, split)
85         training_stats.update({'test_pred': [], 'test_labels': [], '
            test_loss': [], 'test_SHAPGED': []})
86     else:
87         ValueError("split parameter in training must be of length 2
            or 3")
88
89     if not checkpoint:
90         checkpoint = 1
91
92     train_loader = DataLoader(train_set, batch_size=batch_size,
        shuffle=True, num_workers=8)
93     val_loader = DataLoader(val_set, batch_size=batch_size, shuffle=
        True, num_workers=8)
94     if test_set:
95         test_loader = DataLoader(test_set, batch_size=batch_size,
            shuffle=True, num_workers=8)
96
97     model.to(device)
98
99     criterion = nn.CrossEntropyLoss()
100    optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=1e
        -6)
101
102    iters = 0
103    train_predictions = []
104    train_targets = []
105    total_loss = 0.0
106    total_accuracy = 0.0
107    total_shap_ged = 0.0
108
109    for epoch in tqdm(range(epochs), desc='epochs'):

```

```

110     for inputs, labels in tqdm(train_loader, desc='iters', leave=
111                                False):
112         inputs, labels = inputs.to(device), labels.to(device)
113         optimizer.zero_grad()
114         model.train()
115         outputs = model(inputs)
116         loss = criterion(outputs, labels)
117
118         model.eval() # Doesn't affect gradient and backprop,
119                     # changes dropout behaviour for SHAP calculations
120         shap_values = get_shap(model, inputs, size=33)
121         model.train()
122
123         loss_shap = shap_loss(shap_values, KG, torch.argmax(
124                               outputs.detach().cpu(), dim=1), h)
125         loss *= loss_shap
126         loss.backward()
127         optimizer.step()
128
129         train_predictions.extend(torch.argmax(outputs.detach()
130                                               .cpu(), dim=1).numpy().tolist())
131         train_targets.extend(labels.detach().cpu().numpy().tolist())
132
133         total_loss += loss.item()
134         total_accuracy += get_accuracy(outputs.detach().cpu(),
135                                       labels.detach().cpu())
136
137         sag = get_sag(shap_values, torch.argmax(outputs.detach()
138                                               .cpu(), dim=1), labels.detach().cpu(), topk=5)
139         # print(np.where(sag ==1))
140         total_shap_ged += np.sum(KG != sag)
141
142         iters += 1
143
144         # print(f'\nIter [{iters}], Loss: {loss.item():.4f},
145               Accuracy: {get_accuracy(outputs.detach().cpu(),
146                                       labels.detach().cpu()):.4f}, SHAPGED: {np.sum(KG !=
147                                       sag) }')
148
149         if iters % checkpoint == 0:
150             torch.save(model.state_dict(), f'saved_models/
151                               packet_models_shap1/baseline_iter{iters}.pt')
152
153         if iters % log == 0:
154             avg_loss = total_loss / log
155             avg_accuracy = total_accuracy / log

```

```

146         avg_shap_ged = total_shap_ged / log
147
148         training_stats['train_pred'].append(train_predictions
149         )
150         training_stats['train_labels'].append(train_targets)
151         training_stats['train_loss'].append(avg_loss)
152         training_stats['train_SHAPGED'].append(avg_shap_ged)
153
154         print(f'\nIter Checkpoint [{iters}], Loss: {avg_loss:
155         .4f}, Accuracy: {avg_accuracy:.4f}, SHAPGED: {
156         avg_shap_ged}')
157
158         train_predictions = []
159         train_targets = []
160         total_loss = 0.0
161         total_accuracy = 0.0
162         total_shap_ged = 0.0
163
164         model.eval()
165         # with torch.no_grad():
166         val_iters = 0
167         val_predictions = []
168         val_targets = []
169         total_val_loss = 0.0
170         total_val_accuracy = 0.0
171         total_val_shap_ged = 0.0
172
173         for val_data, val_labels in tqdm(val_loader, desc='
174         val_iters', leave=False):
175             val_data, val_labels = val_data.to(device),
176             val_labels.to(device)
177
178             val_outputs = model(val_data)
179             val_loss = criterion(val_outputs, val_labels)
180
181             shap_values = get_shap(model, val_data, size=33)
182             loss_shap = shap_loss(shap_values, KG,
183             torch.argmax(val_outputs.detach().cpu(), dim
184             =1), h)
185             val_loss *= loss_shap
186
187             val_predictions.extend(torch.argmax(
188             val_outputs.detach().cpu(),dim=1).numpy()
189             .tolist())
190             val_targets.extend(val_labels.cpu().numpy()
191             .tolist())

```

```

183         total_val_loss += val_loss.item()
184         total_val_accuracy += get_accuracy(
185             val_outputs.detach().cpu(), val_labels.detach(
186                 ).cpu())
187
188         sag = get_sag(shap_values, torch.argmax(
189             val_outputs.detach().cpu(), dim=1),
190             val_labels.detach().cpu(), topk=5)
191         # print(np.where(sag ==1))
192         total_val_shap_ged += np.sum(KG != sag)
193
194         val_iters += 1
195
196         if val_iters % log == 0:
197
198             val_loss = total_val_loss / log
199             val_accuracy = total_val_accuracy / log
200             val_shap_ged = total_val_shap_ged / log
201
202             training_stats['val_pred'].append(
203                 val_predictions)
204             training_stats['val_labels'].append(
205                 val_targets)
206             training_stats['val_loss'].append(val_loss)
207             training_stats['val_SHAPGED'].append(
208                 val_shap_ged)
209
210             print(f'\nIters Checkpoint [{iters}],
211                 Validation Loss: {val_loss:.4f}, Accuracy:
212                 {val_accuracy:.4f}, SHAPGED: {
213                 val_shap_ged}')
214             with open(f'training_metrics/
215                 packet_stats_shap1/training_stats_iter{
216                     iters}.json', 'w') as json_file:
217                 json.dump(training_stats, json_file)
218             break
219
220     if test_set:
221         model.eval()
222         test_predictions = []
223         test_targets = []
224         total_test_loss = 0.0
225         total_test_accuracy = 0.0
226         total_test_shap_ged = 0.0
227
228         # with torch.no_grad():
229         for test_data, test_labels in test_loader:

```



```

217         test_data, test_labels = test_data.to(device),
           test_labels.to(device)
218
219         test_outputs = model(test_data)
220         test_loss = criterion(test_outputs, test_labels)
221
222         shap_values = get_shap(model, test_data, size=33)
223         loss_shap = shap_loss(shap_values, KG, torch.argmax(
           test_outputs.detach().cpu(), dim=1), h)
224         test_loss *= loss_shap
225
226         test_predictions.extend(torch.argmax(test_outputs.detach
           ().cpu(), dim=1).numpy().tolist())
227         test_targets.extend(test_labels.cpu().numpy().tolist())
228
229         total_test_loss += test_loss.item()
230         total_test_accuracy += get_accuracy(test_outputs.detach()
           .cpu(), test_labels.detach().cpu())
231
232         sag = get_sag(shap_values, torch.argmax(
           val_outputs.detach().cpu(), dim=1), val_labels, 0.5)
233         total_test_shap_ged += np.sum(KG != sag)
234
235         test_loss = total_test_loss / len(test_loader)
236         test_accuracy = total_test_accuracy / len(test_loader)
237         test_shap_ged = total_test_shap_ged / len(test_loader)
238
239         training_stats['test_pred'] = test_predictions
240         training_stats['test_labels'] = test_targets
241         training_stats['test_loss'] = test_loss
242         training_stats['test_SHAPGED'] = test_shap_ged
243
244         print(f'Epoch [{epoch + 1}/{epochs}], Test Loss: {test_loss:
           .4f}, Accuracy: {test_accuracy:.4f}, SHAPGED: {
           test_shap_ged}')
245
246         torch.save(model.state_dict(), 'saved_models/packet_models_shap1/
           baseline_final.pt')
247         with open('training_metrics/packet_stats_shap1/
           training_stats_final.json', 'w') as json_file:
248             json.dump(training_stats, json_file)
249
250
251 if __name__ == '__main__':
252     seed = 774
253
254     # Hyperparameters

```

```

255     batch_size = 256
256     learning_rate = 0.0001
257     epochs = 10 # 500
258
259     # with open('data/utils/filters.json','r') as f:
260     #     filters = json.load(f)
261
262     KG = pd.read_csv('knowledge_graphs/KG_2.csv').to_numpy()[ :,1:]
263
264     data_dir = "data/Processed Data/"
265     ddos_dataset = CICDDoS2019Dataset(data_dir, transform=F.normalize
266         , transform_args=[2.0,0], balanced=True, random_seed=seed)
267
268     model = BaselineCNN(channel_size=ddos_dataset.data.shape[-2])
269     # state_dict = torch.load('saved_models/packet_models/
270     #     baseline_final.pt')
271     # model.load_state_dict(state_dict)
272     run_shap_training(model, ddos_dataset, epochs, KG, batch_size=
273         batch_size, lr=learning_rate, checkpoint=50, log=20, h=0.01)

```

A.2 Dataset Code

```

1  import os
2  from pathlib import *
3  import random
4
5  import numpy as np
6  import pandas as pd
7  import torch
8  from torch.utils.data import Dataset
9  from tqdm import tqdm
10
11  class CICDDoS2019Dataset(Dataset):
12      def __init__(self, dir_path:str, seq_length:int=1, transform=None
13          ,
14              transform_args:list[any]=None, filters:list[str]=
15              None, binary:bool=False,
16              balanced:bool=False, sample_sizes:int|float|dict[int
17              ]|dict[float]=None, random_seed:int=None):
18
19          self.transform = transform
20          self.t_args = transform_args
21          self.sequence_features = True if seq_length > 1 else False
22
23          if random_seed:
24              np.random.seed(random_seed)

```

```

22     size_map = {}
23     for f_path in tqdm(Path(dir_path).iterdir(), desc='
24         allocating...'):
25         if os.path.isfile(f_path):
26             num_lines = sum(1 for _ in open(f_path, "rb"))
27             size_map[str(f_path)] = num_lines - 1
28
29     if sample_sizes:
30         match sample_sizes:
31             case int():
32                 self.sample_sizes = dict.fromkeys(size_map.keys()
33                     ,sample_sizes)
34             case float():
35                 self.sample_sizes = {key: int(value *
36                     sample_sizes) for key, value in size_map.items()
37                     ()}
38             case dict():
39                 assert set(sample_sizes.keys()) == set(
40                     size_map.keys())
41                 if all(isinstance(value, int) for value in
42                     sample_sizes.values()):
43                     self.sample_sizes = sample_sizes
44                 elif all(isinstance(value, float) for value in
45                     sample_sizes.values()):
46                     self.sample_sizes = {key: int(value *
47                         sample_sizes[key]) for key, value in
48                         size_map.items()}
49                 else:
50                     TypeError('invalid sample_size dict')
51             case _:
52                 TypeError('sample_sizes needs to be of type int,
53                     float, dict[int], or dict[float]')
54         for key in self.sample_sizes.keys():
55             if self.sample_sizes[key] > size_map[key]:
56                 self.sample_sizes[key] = size_map[key]
57             assert self.sample_sizes[key] <= size_map[key]
58
59     if balanced and not sample_sizes:
60         min_size = min(size_map.values())
61         self.sample_sizes = dict.fromkeys(size_map.keys(),
62             min_size)
63     if binary:
64         for key, value in self.sample_sizes.items():
65             if 'Benign' not in key:
66                 self.sample_sizes[key] = int(value / (len(
67                     self.sample_sizes)-1))

```

```

57
58     if self.sequence_features:
59         for key in self.sample_sizes.keys():
60             if self.sample_sizes[key] % seq_length != 0:
61                 self.sample_sizes[key] -= self.sample_sizes[key]
62                 % seq_length
63                 assert self.sample_sizes[key] % seq_length == 0
64
65     dfs = []
66     for f_path in tqdm(Path(dir_path).iterdir(), desc='collecting
67         data...'):
68         if os.path.isfile(f_path):
69             negative_sample = size_map[str(f_path)] -
70             self.sample_sizes[str(f_path)]
71             skip_index = random.sample(range(1, size_map[str(
72                 f_path)]), negative_sample) if negative_sample > 0
73             else None
74             df = pd.read_csv(f_path, skiprows=skip_index, usecols
75                 =filters)
76             dfs.append(df)
77             if self.sequence_features:
78                 assert(len(dfs[-1]) % seq_length == 0)
79
80     df = pd.concat(dfs)
81
82     data = torch.tensor(df.iloc[:, :-1].values, dtype=
83         torch.float32)
84     labels = torch.tensor(df.iloc[:, -1].values, dtype=torch.long)
85     if binary:
86         mask = labels > 0
87         labels[mask] = 1
88
89     if self.transform:
90         if self.t_args:
91             data = self.transform(data,*self.t_args)
92             data = self.transform(data)
93
94     if self.sequence_features:
95         data = torch.reshape(data, (-1,seq_length, data.shape[1])
96             ) # n, _ , features
97         self.data = torch.permute(data,(0,2,1)) # n,
98             sequence_length , features -> n, channels(features),
99             sequence_length | conv over sequence
100         labels = torch.reshape(labels, (-1,seq_length))
101
102     for sequence_labels in tqdm(labels[:,], desc='verifying
103         sequence labels...'):
104         same = torch.all(torch.eq(sequence_labels,

```

```

        sequence_labels[0]))
        assert(same)
        self.labels = labels[:,0]
    else:
        self.data = data.unsqueeze(1) # n, channel(1), feature |
        conv over packet features
        self.labels = labels
98
99
100 def __len__(self):
101     return len(self.data)
102
103 def __getitem__(self, idx):
104     return self.data[idx], self.labels[idx]
105
106 def min_max_normalize(data:torch.Tensor, dim:int=1, new_min:float=0,
    new_max:float=1):
107     normed = data
108     normed -= normed.min(dim, keepdim=True)[0]
109     normed /= normed.max(dim, keepdim=True)[0]
110     normed *= (new_max - new_min)
111     normed += new_min
112     return normed

```

A.3 Anomaly Detection Model Code

```

1 import torch.nn as nn
2
3 class BinaryCNN(nn.Module):
4     def __init__(self, channel_size:int):
5         super(BinaryCNN, self).__init__()
6         kernel1D_sizes = [3,3]
7         pooling_kernel_size = 2
8         pooling_stride = 2
9
10        # Input layer
11        self.c1d1 = nn.Conv1d(in_channels=channel_size, out_channels
            =64, kernel_size=kernel1D_sizes[0])
12        self.c1d_af1 = nn.ReLU()
13
14        self.c1d2 = nn.Conv1d(in_channels=64, out_channels=64,
            kernel_size=kernel1D_sizes[1])
15        self.c1d_af2 = nn.ReLU()
16
17        # Convolutional layer

```

```

18         self.c1d_pool = nn.MaxPool1d(kernel_size=pooling_kernel_size,
19                                         stride=pooling_stride)
20
21     # Dense layers
22     self.dropout = nn.Dropout(0.5)
23     self.fc1 = nn.Linear(2048, 128)
24     self.fc_af1 = nn.ReLU()
25     self.fc2 = nn.Linear(128, 2)
26
27     def forward(self, x):
28         # Input layer
29         x = self.c1d1(x)
30         x = self.c1d_af1(x)
31
32         # Convolutional layer
33         x = self.c1d2(x)
34         x = self.c1d_af2(x)
35         x = self.c1d_pool(x)
36
37         # Flatten for the dense layers
38         x = x.view(x.size(0), -1)
39         x = self.dropout(x)
40         # Dense layers
41         x = self.fc1(x)
42         x = self.fc_af1(x)
43         x = self.fc2(x)
44
45         return x

```

A.4 Signature Detection Model Code

```

1 import torch.nn as nn
2
3 class BaselineCNN(nn.Module):
4     def __init__(self, channel_size:int):
5         super(BaselineCNN, self).__init__()
6         kernel1D_sizes = [3,3,5]
7         pooling_kernel_size = 2
8         pooling_stride = 2
9
10        # Input layer
11        self.c1d1 = nn.Conv1d(in_channels=channel_size, out_channels
12                                =64, kernel_size=kernel1D_sizes[0])
13        self.c1d_af1 = nn.ReLU()

```

```

14         self.c1d2 = nn.Conv1d(in_channels=64, out_channels=64,
15                                kernel_size=kernel1D_sizes[1])
16         self.c1d_af2 = nn.ReLU()
17
18         # Convolutional layer
19         self.c1d3 = nn.Conv1d(in_channels=64, out_channels=64,
20                                kernel_size=kernel1D_sizes[2])
21         self.c1d_af3 = nn.ReLU()
22         self.c1d_pool = nn.MaxPool1d(kernel_size=pooling_kernel_size,
23                                       stride=pooling_stride)
24
25         # Dense layers
26         self.dropout = nn.Dropout(0.5)
27         self.fc1 = nn.Linear(1920, 256)
28         self.fc_af1 = nn.ReLU()
29         self.fc2 = nn.Linear(256, 128)
30         self.fc_af2 = nn.ReLU()
31         self.fc3 = nn.Linear(128, 13)
32
33     def forward(self, x):
34         # Input layer
35         x = self.c1d1(x)
36         x = self.c1d_af1(x)
37
38         # Convolutional layer
39         x = self.c1d2(x)
40         x = self.c1d_af2(x)
41         x = self.c1d3(x)
42         x = self.c1d_af3(x)
43         x = self.c1d_pool(x)
44
45         # Flatten for the dense layers
46         x = x.view(x.size(0), -1)
47         x = self.dropout(x)
48         # Dense layers
49         x = self.fc1(x)
50         x = self.fc_af1(x)
51         x = self.fc2(x)
52         x = self.fc_af2(x)
53         x = self.fc3(x)
54
55         return x

```

Appendix B

Additional Machine Learning Content

B.1 Convolutional Neural Networks

Convolutional Neural Networks (CNN) serve as powerful tools in the field of artificial intelligence, particularly in the domain of computer vision and sequence analysis. Structured as a feed-forward neural network, a CNN employs a combination of fully connected dense layers, convolutional layers, and pooling layers to process and distill information critical to its classification task. [4]

The architecture of a CNN involves fully connected dense layers, where neurons are organized in an array with assigned weights based on activation functions and input values received from the preceding layer or input features. Convolutional layers, crucial to CNNs, extract significant features from a data tensor through the use of receptive fields per neuron. This is achieved by employing a convolutional kernel with trainable weights that shift along inputs to convolve them and fit the receptive field of each neuron. Pooling layers, another integral component, reduce data dimensionality by consolidating a cluster of neurons from the input layer into a single neuron, accomplished through methods such as max pooling or average pooling. [4]

While CNNs are conventionally applied to 2D computer vision tasks, their versatility extends to 1D applications for sequence analysis. In this context, 1D CNNs prove more efficient than traditional Recurrent Neural Network (RNN) architectures, circumventing issues like the vanishing or exploding gradient problem through the implementation of regularization methods. [5]

The applicability of CNNs is particularly relevant in the context of IDS data packet analysis, which fundamentally involves a classification task on time series sequence data. Leveraging the strengths of CNNs, especially in handling sequential data efficiently, makes them a compelling choice for training IDS, offering a promising avenue for enhancing cybersecurity through explainable AI.

B.2 CICDDoS2019 Features

Table B.1: Raw Dataset Features and Descriptions

Feature Name	Description
Flow Duration	Duration of the flow in Microsecond
Total Fwd Packets	Total packets in the forward direction
Total Bwd Packets	Total packets in the backward direction
Total Length of Fwd Packet	Total size of packet in forward direction
Total Length of Bwd Packet	Total size of packet in backward direction
Fwd Packet Length Min	Minimum size of packet in forward direction
Fwd Packet Length Max	Maximum size of packet in forward direction
Fwd Packet Length Mean	Mean size of packet in forward direction
Fwd Packet Length Std	Standard deviation size of packet in forward direction
Bwd Packet Length Min	Minimum size of packet in backward direction
Bwd Packet Length Max	Maximum size of packet in backward direction
Bwd Packet Length Mean	Mean size of packet in backward direction
Bwd Packet Length Std	Standard deviation size of packet in backward direction
Flow Bytes/s	Number of flow bytes per second
Flow Packets/s	Number of flow packets per second
Flow IAT Mean	Mean time between two packets sent in the flow
Flow IAT Std	Standard deviation time between two packets sent in the flow
Flow IAT Max	Maximum time between two packets sent in the flow
Flow IAT Min	Minimum time between two packets sent in the flow
Fwd IAT Min	Minimum time between two packets sent in the forward direction
Fwd IAT Max	Maximum time between two packets sent in the forward direction
Fwd IAT Mean	Mean time between two packets sent in the forward direction
Fwd IAT Std	Standard deviation time between two packets sent in the forward direction
Fwd IAT Total	Total time between two packets sent in the forward direction
Bwd IAT Min	Minimum time between two packets sent in the backward direction
Bwd IAT Max	Maximum time between two packets sent in the backward direction

Feature Name	Description
Bwd IAT Mean	Mean time between two packets sent in the backward direction
Bwd IAT Std	Standard deviation time between two packets sent in the backward direction
Bwd IAT Total	Total time between two packets sent in the backward direction
Fwd PSH Flags	Number of times the PSH flag was set in packets travelling in the forward direction (0 for UDP)
Bwd PSH Flags	Number of times the PSH flag was set in packets travelling in the backward direction (0 for UDP)
Fwd URG Flags	Number of times the URG flag was set in packets travelling in the forward direction (0 for UDP)
Bwd URG Flags	Number of times the URG flag was set in packets travelling in the backward direction (0 for UDP)
Fwd Header Length	Total bytes used for headers in the forward direction
Bwd Header Length	Total bytes used for headers in the backward direction
Fwd Packets/s	Number of forward packets per second
Bwd Packets/s	Number of backward packets per second
Packet Length Min	Minimum length of a packet
Packet Length Max	Maximum length of a packet
Packet Length Mean	Mean length of a packet
Packet Length Std	Standard deviation length of a packet
Packet Length Variance	Variance length of a packet
FIN Flag Count	Number of packets with FIN
SYN Flag Count	Number of packets with SYN
RST Flag Count	Number of packets with RST
PSH Flag Count	Number of packets with PUSH
ACK Flag Count	Number of packets with ACK
URG Flag Count	Number of packets with URG
CWE Flag Count	Number of packets with CWR
ECE Flag Count	Number of packets with ECE
Down/Up Ratio	Download and upload ratio
Average Packet Size	Average size of packet
Avg Fwd Segment Size	Average size observed in the forward direction
Avg Bwd Segment Size	Average size observed in the backward direction
Fwd Avg Bytes/Bulk	Average number of bytes bulk rate in the forward direction
Fwd Avg Packet-s/Bulk	Average number of packets bulk rate in the forward direction
Fwd Avg Bulk Rate	Average number of bulk rate in the forward direction
Bwd Avg Bytes/Bulk	Average number of bytes bulk rate in the backward direction
Bwd Avg Packet-s/Bulk	Average number of packets bulk rate in the backward direction

Feature Name	Description
Bwd Avg Bulk Rate	Average number of bulk rate in the backward direction
Subflow Fwd Packets	The average number of packets in a sub flow in the forward direction
Subflow Fwd Bytes	The average number of bytes in a sub flow in the forward direction
Subflow Bwd Packets	The average number of packets in a sub flow in the backward direction
Subflow Bwd Bytes	The average number of bytes in a sub flow in the backward direction
Fwd Init Win Bytes	The total number of bytes sent in initial window in the forward direction
Bwd Init Win Bytes	The total number of bytes sent in initial window in the backward direction
Fwd Act Data Pkts	Count of packets with at least 1 byte of TCP data payload in the forward direction
Fwd Seg Size Min	Minimum segment size observed in the forward direction
Active Min	Minimum time a flow was active before becoming idle
Active Mean	Mean time a flow was active before becoming idle
Active Max	Maximum time a flow was active before becoming idle
Active Std	Standard deviation time a flow was active before becoming idle
Idle Min	Minimum time a flow was idle before becoming active
Idle Mean	Mean time a flow was idle before becoming active
Idle Max	Maximum time a flow was idle before becoming active
Idle Std	Standard deviation time a flow was idle before becoming active

Appendix C

Relevant Explainable Machine Learning Material

C.1 Local Interpretable Model Agnostic Explanations

LIME was first proposed by Ribeiro et al. in "*Why Should I Trust You?: Explaining the Predictions of Any Classifier*" [15] for the purpose of explaining model predictions of any classifier in a human-interpretable manner. LIME employs an interpretable surrogate model, and trains it to estimate the predictions of a black box model for a predetermined set of data points. To achieve this, they utilize visual or textual artifacts that are easily interpretable by humans. LIME surrogates are specifically chosen to be locally faithful and trained to match individual predictions that align with those of the black box model. [15] Examples of such interpretable models include linear, k-nearest neighbor, Naive Bayes, decision tree, and RuleFit models and their respective variants. Interpretable surrogates rely on the presence or absence of specific features to predict the decision boundaries of black box models. The surrogate models in LIME are designed to strike a balance between fidelity and interpretability. This balance is articulated as a trade-off, signifying that as surrogate models aim for greater accuracy in approximating the black box model, their complexity tends to increase, making them less interpretable.

The process of developing a local surrogate model explanation are expressed mathematically in the form of equation (C.1) [15] Where ξ represents the optimization of the surrogate model. $\mathcal{L}$ represents the fidelity function, and decreases when the surrogate model's prediction g approaches that of the black box model's prediction f , based on a proximity measure π_x . Note that z' must first be transformed back to its original feature space through $h_x(z') = z$ to be representative of the original data f predicted on. The fidelity function is balanced by a regularizer in the surrogate model complexity measure Ω , which increases based on the complexity of g , which must be defined a priori.

$$\xi(x) = \arg \min_{g \in G} \mathcal{L}(f, g, \pi_x) + \Omega(g) \quad (\text{C.1})$$

$$\mathcal{L}(f, g, \pi_x) = \sum_{z' \in \mathcal{Z}} \pi_x(z') (f(h_x(z')) - g(z'))^2 \quad (\text{C.2})$$

The selective and contrastive approach presented by LIME makes it seem like a compelling choice in explaining NIDS predictions, however there are several weaknesses with this method of analysis. The primary issue with LIME has to be the instability of explanations when the observed data does not reflect reality. This is especially true, as surrogate models are usually fed data sampled via proximity to true data in a Gaussian distribution, whereas data such as network traffic generated from human behaviour often follows a power law distribution. The Ω complexity parameter also poses a considerable problem, as it can heavily influence the surrogate model’s prediction if chosen poorly. [16] Given these limitations, there exist no notable papers utilising the LIME method to evaluate and explain the decisions of NIDS. However, it has seen use in fields with similar dimensional data in natural language processing and sequence analysis tasks. The fundamental concepts of LIME are also critical components in the understanding of SHAP, a significantly more popular method for xAI in the cybersecurity space.

C.2 Shapley Values

Shapley values borrow directly from Lloyd Shapley et al.’s *A Value For N-Person Games*[7], which proposes an algorithm to determine the quantitative contribution of players to a cooperative game with a unified goal between its players. Shapley’s idea is repurposed in the context of xAI and used to determine the contribution each of each feature towards a prediction for a given vector of feature values. This process is then repeated over all possible feature value combinations. [7]

$$\phi_j(val) = \sum_{S \subseteq \{1, \dots, p\} \setminus \{j\}} \frac{|S|!(p - |S| - 1)!}{p!} (val(S \cup \{j\}) - val(S)) \quad (C.3)$$

Equation (C.3) demonstrates the calculation of the Shapley value for feature j , where S is the subset of features the model was trained on, and p is the number of features. [16]

$$val_x(S) = \int \hat{f}(x_1, \dots, x_p) d\mathbb{P}_{x \notin S} - E_x(\hat{f}(X)) \quad (C.4)$$

Equation (C.4) represents the value function for the prediction of data point x with feature values in set S that are marginalized over features that are not included in S relative to the expected value of the black box’s prediction of the data point x . [16]

The benefit of the mathematical robustness provided by Shapley values is a very solid theory and explanation; Shapley values have their own axioms of efficiency, symmetry, dummy, and additivity. Efficiency states that feature contributions must total to the difference in the prediction of x and the expected result of the black box. [16]

$$\sum_{j=1}^p \phi_j = \hat{f}(x) - E_X(\hat{f}(X)) \quad (C.5)$$

Symmetry suggests that the contribution and therefore Shapley values of two features j and k are the same $\phi_j = \phi_k$ if they contribute equally to all possible coalition combinations $val(S \cup \{j\}) = val(S \cup \{k\})$ for $S \subseteq \{1, \dots, p\} \setminus \{j, k\}$. [16]

The dummy and additivity axioms are mathematically trivial under the assumption of a Bayesian feature set, where the value of each feature is independent of other features in its subset. The dummy axiom states that a feature with a Shapley value of 0 has no impact on the predicted value when added to a coalition. The additivity axiom states that for aggregate predictions using the same feature j , the Shapley values ϕ_j can be summed across predictions.

This advantage comes at a significant computational cost. For a proper and accurate measure using Shapley values, one would see that the integral be performed repeatedly over all features $x \notin S$, which grows the computation exponentially for each feature excluded from the subset. [16] This problem has led xAI researchers to estimation methods to approximate the Shapley value of features for large datasets and models. This is done through Monte-Carlo sampling. Where the features of S are selected such that the law of averages would successfully approximate S when the operation is performed for each feature to get all Shapley values. [16] This limitation on computation leading to simplification of input data in addition to being a permutation-based interpretation method sensitive to data that does not reflect reality makes it a sub-optimal candidate to explain IDS systems, however its mathematical foundation can be merged into other methods to accumulate an additional layer of confidence in an explanation.

Despite these improvements over LIME, most recent papers in the xAI and cybersecurity domain use the following SHAP papers to dissect model decision boundaries, as SHAP has generally yielded more accurate results and comprehensive explanations in adjacent sequence analysis tasks.

C.3 SHapley Additive exPlanations

SHAP bridges the LIME and Shapley value analysis methods to create a surrogate linear system using the Shapley value contributions of each of the weights. This guarantees the optimization of the surrogate model’s prediction towards the black box’s prediction to be dependent on the contribution of each feature to the predictive class. [14]

$$g(z') = \phi_0 + \sum_{j=1}^M \phi_j z'_j \quad (\text{C.6})$$

Equation (C.6) illustrates the relation of the explanation model prediction g , the series of M Shapley values ϕ and a coalition vector $z \in \{0, 1\}^M$ which determines if a feature’s contribution effects the model’s prediction.[14] The mathematical properties of Shapley values (efficiency, dummy, symmetric, additive) carry over to that of SHAP, as it is a linear system of Shapley values abiding by the additive axiom. In addition to Shapley axioms, SHAP has its own properties including local accuracy, missingness, and consistency. Local accuracy is guaranteed when $\phi_0 = E_x(f(x))$ and $z' = \mathbf{1}$ as SHAP degrades back to the efficiency property of Shapley values. Missingness is guaranteed by the definition of z' as contributions of absent features to the prediction are nullified. Consistency is guaranteed by the linearity of SHAP, as contributions from features are always 1st degree meaning increases are consistent across models but not necessarily uniform. [16]

Given its mathematical soundness and flexibility, SHAP has seen wide adoption and use across all applications of the xAI field, including IDS within the realm of cybersecurity. Prior works use SHAP as a post hoc analysis tool to examine the results of IDS classifications with reasonable human interpretable explanation for a subset of features. SHAP is even perceived as reliable enough to provide insight as a data analytics tool like principle component analysis (PCA). [16] This makes SHAP a reasonable candidate to further improve the efficiency and construct upon to improve model explanation and approximations.

Most results from studies utilising SHAP for analysing CNN architectures in IDS find similar result distributions depending on the dataset the CNN was trained on. [1] The permutation of features selected differs from study to study based on researcher biases and the cyber attack of focus, however, it is often true that the Shapley values used to construct the linear system follow a power law distribution in terms of contribution to the prediction. It would be safe to say that this trend is an expected result of synthetic data. [1]

C.4 eXplainable Neural-Symbolic Learning

EXplainable Neural-Symbolic Learning (X-NeSyL) is a noteworthy exploration within the field of Symbolic AI, demonstrating its substantial contributions to Explainable AI. In the context of Neuro-Symbolic methods, researchers integrate prior human knowledge into tasks like concept learning, yielding more interpretable outputs, such as mathematical equations or Domain-Specific Languages (DSL). [13] The X-NeSyL approach specifically leverages symbolic knowledge from domain experts, expressed as a knowledge graph (KG), comparing it with explanations generated by a neural network. The method targets the enhancement of the neural network’s interpretability by encouraging it to assign values of feature importance to the properties of data that reflect human compositional classification. X-NeSyL functions as an extension of state-of-the-art CNN architectures and methods like SHAP for researchers construct a graph embodying constraints and relations derived from computed feature importance values relative to an expert-provided KG. The incorporation of a designated loss function penalizes non-overlap between the two, demonstrating improvements in both explainability which can lead researchers to an increase in performance in future model architectures. [13]

X-NeSyL is explainable-by-design as it is modelled specifically with an expertly developed KG in scope, ensuring interpretability at each stage of the training process. Unlike ad-hoc methods, this approach integrates a loss function to guide the neural network toward generating explanations structured akin to human expertise. [13] A notable limitation arises from the method’s dependence on domain-specific knowledge, which may be challenging to obtain from experts and might introduce contradictions when multiple experts are involved, despite the benefits yielded by the effort and use of SHAP values produces straightforward intermediate feature relevance results. Prior works acknowledge this challenge, and as a result conduct extensive testing with various datasets and CNN architectures, assessing performance results alongside established and novel methods to discern the practical efficacy of the human-in-the-loop approach proposed by X-NeSyL. [16]

There were no prior works which applied the method of X-NeSyL to IDS or known IDS datasets, likely because of its higher barrier to entry in terms of development with the

requirement of developing a KG from domain experts. The benefits of this method may also not be as significant when applied to tabular data, as prior works regarding this method utilise primarily various forms of image data which is treated in a fundamentally different way than tabular data and sequence data which NIDS data packet analysis resembles. The method is still worth investigating nonetheless, as its SHAP and CNN requirements are likely core components of the study and will be completed regardless of X-NeSyL’s prospective performance when used for DDoS anomaly detection in NIDS.

